

RandomDataStreams.jl

September 3, 2026

Contents

Contents	ii
I Home	1
1 RandomDataStreams.Jl Documentation	2
1.1 Contents	2
1.2 Quick example	3
II Getting Started	4
2 Getting Started	5
2.1 Installation	5
2.2 Choosing a generator	5
2.3 Seeding: one rule for every generator	5
2.4 First numbers	6
2.5 Supported output types	7
2.6 Reproducibility	7
2.7 Full Random API	7
2.8 Next steps	8
III Streams & Substreams	9
3 Streams & Substreams	10
3.1 Why streams? The case for Common Random Numbers (CRN)	10
3.2 The two object families	12
3.3 Producing independent streams	12
3.4 Threads	12
3.5 Distributed	13
3.6 Navigating substreams	14
3.7 Saving, restoring, jumping	14
3.8 Scope: host-side streams, device-side bijections	15
3.9 One interface, every generator	15
3.10 Guarantees	16
IV API Reference	17
4 API Reference	18
4.1 Types	18
4.2 The common interface	18
4.3 MRG32k3a	19
4.4 MRG63k3a	20
4.5 Xoshiro256+	22
4.6 Xoshiro / xoroshiro families	22
4.7 Where the families actually differ	23
4.8 PCG	23

4.9	Counter-based generators	25
V	Docstrings	28
5	Docstrings	29
5.1	Abstract types	29
5.2	MRG32k3a	30
5.3	MRG63k3a	31
5.4	Xoshiro / xoroshiro families	31
5.5	Counter-based generators	35
VI	Implementation Notes	44
6	Implementation Notes	45
6.1	MRG32k3a	45
6.2	MRG63k3a	46
6.3	Xoshiro256+	48
6.4	PCG	49
6.5	Counter-based generators	50
6.6	Integer outputs of MRG32k3a	50
6.7	Testing strategy	51
6.8	Performance snapshot	51
VII	Validation	54
7	Statistical Validation (TestU01)	55
7.1	In the test suite: SmallCrush	55
7.2	Dependence between streams	55
7.3	The integer paths, at the bit level	56
7.4	Running Crush and BigCrush	56
7.5	Long campaigns: running for days without a session	57
VIII	Generator Comparison	60
8	Choosing a generator	61
8.1	Recurrence-based generators	61
8.2	Counter-based generators	61
8.3	When to choose MRG32k3a	61
8.4	When to choose MRG63k3a	62
8.5	When to choose a xoshiro/xoroshiro variant	63
8.6	When to choose PCG64	63
8.7	When to choose Philox or Threefry	64
8.8	Scrambler choice within a xoshiro family	64
8.9	Related work in the Julia ecosystem	64
8.10	Practical notes	65
IX	FAQ	66
9	FAQ	67
9.1	Why is an all-zero state forbidden?	67
9.2	Are MRG integer outputs uniform over their full width?	67
9.3	MRG32k3a or MRG63k3a?	67
9.4	Why doesn't <code>sample(v, k)</code> work with <code>RandomDataStreams</code> generators?	67
9.5	Where did <code>srand</code> , <code>next_stream</code> , <code>short_jump</code> , <code>long_jump</code> go?	68
9.6	What does an integer seed mean?	68
9.7	How do I run truly parallel simulations?	68
9.8	Which generator should I pick?	68

9.9	What does "counter-based" buy me?	68
9.10	Philox or Threefry?	68
9.11	Is it safe to use 1, 2, 3, ... as counter-based stream keys?	69
9.12	Can I reproduce a NumPy pipeline?	69
9.13	Why can't I choose PCG's increment to get more streams?	69
9.14	Does this package run on the GPU?	69

Part I

Home

Chapter 1

RandomDataStreams.jl Documentation

Streamable pseudo-random number generators for Julia, with non-overlapping streams and substreams:

- **MRG32k3a** (L'Ecuyer et al. 2002) — the reference combined MRG of the simulation literature;
- **MRG63k3a** (L'Ecuyer 1999) — the same construction in 64-bit arithmetic: period $\approx 2^{377}$, and a step that yields 63 random bits instead of 32;
- the **xoshiro** / **xoroshiro** families (Blackman & Vigna) — 128, 256 and 512 bits of state, three scramblers each;
- **PCG64** and **PCG64DXSM** (O'Neill) — the default bit generator of NumPy, matched bit for bit, with streams from the closed-form LCG jump;
- the **counter-based** generators **Philox** and **Threefry** (Salmon et al.
 1. — streams are keys, and any draw can be recomputed from its index.

All of them share one interface: the same stream and substream navigation, the same state contract, and the full Random API.

1.1 Contents

1. [Getting Started](#)
2. [Streams & Substreams](#)
3. [API Reference](#)
4. [Docstrings](#)
5. [Implementation Notes](#)
6. [Validation](#)
7. [Generator Comparison](#)
8. [FAQ](#)

1.2 Quick example

```
using RandomDataStreams

gen = MRG32k3aGen()
rngA = next_stream!(gen)      # independent stream A
rngB = next_stream!(gen)      # independent stream B

rand(rngA)                   # Float64 in [0, 1)
next_substream!(rngA)         # move to the next substream of A
reset_stream!(rngA)           # rewind stream A to its beginning
```

The same code runs on any generator the package ships — only the first line changes:

```
gen = Philox4x64Gen()        # or Xoshiro256plusGen(), Threefry4x64Gen(), ...
```

Part II

Getting Started

Chapter 2

Getting Started

RandomDataStreams.jl provides pseudo-random number generators with support for **non-overlapping streams and substreams**, following the stream/substream model of L'Ecuyer et al. (2002).

2.1 Installation

```
using Pkg
Pkg.add("RandomDataStreams")
```

or, from a local clone:

```
using Pkg
Pkg.develop(path = "path/to/RandomDataStreams.jl")
```

Requirements: Julia ≥ 1.6 . The only dependency is the standard library Random.

2.2 Choosing a generator

RandomDataStreams.jl implements the full xoshiro/xoroshiro family of Blackman & Vigna, L'Ecuyer's MRG32k3a and MRG63k3a, O'Neill's PCG (the default bit generator of NumPy), and the Philox and Threefry CBRNGs. All variants of a xoshiro family share the same linear transition and jump constants; only the output scrambler differs.

See [Generator Comparison](#) for guidance and benchmarks.

2.3 Seeding: one rule for every generator

Whatever generator you pick, there are three ways to seed it, and they mean the same thing in every family:

```
MRG32k3aGen()           # the package default seed, 12345
MRG32k3aGen(20260830)    # an integer seed
MRG32k3aGen([1,2,3,4,5,6]) # the family's own seed vector

Xoshiro256ppGen(20260830) # exactly the same three forms
PCG64Gen(20260830)
PhiloxGen(20260830)
```

Type	Family	State	Period	Scrambler
PhiloxRNG	Philox4x32-10	128-bit counter	2^{130}	Feistel-like network
Philox4x64RNG	Philox4x64-10	128-bit counter	2^{130}	Feistel-like network
Threefry4x64RNG	Threefry4x64-20	128-bit counter	2^{130}	Threefish rounds (add/rot/xor)
Threefry4x32RNG	Threefry4x32-20	128-bit counter	2^{130}	Threefish rounds (add/rot/xor)
Xoroshiro128p	xoroshiro128	128 bits	$2^{128} - 1$	$s_0 + s_1$
Xoroshiro128ss	xoroshiro128	128 bits	$2^{128} - 1$	high bits, **
Xoroshiro128pp	xoroshiro128	128 bits	$2^{128} - 1$	high bits, ++
Xoshiro256p	xoshiro256	256 bits	$2^{256} - 1$	$s_0 + s_3$
Xoshiro256ss	xoshiro256	256 bits	$2^{256} - 1$	high bits, **
Xoshiro256pp	xoshiro256	256 bits	$2^{256} - 1$	high bits, ++
Xoshiro512p	xoshiro512	512 bits	$2^{512} - 1$	$s_0 + s_2$
Xoshiro512ss	xoshiro512	512 bits	$2^{512} - 1$	high bits, **
Xoshiro512pp	xoshiro512	512 bits	$2^{512} - 1$	high bits, ++
MRG32k3a	combined MRG (L'Ecuyer)	192 bits	$\approx 2^{191}$	native Float64, 32-bit step
MRG63k3a	combined MRG (L'Ecuyer)	384 bits	$\approx 2^{377}$	native Float64, 63-bit step
PCG64	PCG (O'Neill)	128 bits	2^{128}	XSL-RR permutation
PCG64DXSM	PCG (O'Neill)	128 bits	2^{128}	DXSM permutation

An **integer** is a *seed*: it is expanded through `splitmix64` and folded into whatever the family accepts, so `T(20260830)` is equivalent to `Random.seed!(T(), 20260830)` everywhere. A value in the family's **own representation** — its seed vector, or a `UInt128` for PCG — is taken as the state or key itself, not hashed.

The same three forms work on the stream types directly (`MRG32k3a(20260830)`, `Xoshiro256pp(20260830)`, ...). See [Streams & Substreams](#) for the full interface table.

2.4 First numbers

```
using RandomDataStreams
```

```
rng = MRG32k3a()      # default seed [12345, ..., 12345]
rand(rng)              # 0.12701112204657714
rand(rng)              # 0.3185275653967945
```

```
x = Xoshiro256p(UInt64[1, 2, 3, 4])
rand(x)                # Float64 in [0, 1)
```

2.5 Supported output types

MRG32k3a

- Float64 (native), plus Float32, Float16 via conversion
- UInt8, UInt16, UInt32, UInt64, UInt128
- Int8, Int16, Int32, Int64, Int128 (same bit patterns)
- Bool
- Ranges: `rand(rng, 1:10)` works through the standard Random machinery

Note: integer outputs of MRG32k3a are built from its native ~32-bit resolution, and MRG63k3a's from its ~63-bit one; they are not uniform over their full width. Use them for indices, choices and flags rather than cryptography.

Xoshiro256p

- Float64 (native path), Float32, Float16
- UInt64
- Ranges: `rand(rng, 1:10)` works through the standard Random machinery

2.6 Reproducibility

Every generator accepts an explicit seed:

```
rng = MRG32k3a([42, 1, 2, 3, 4, 5])
x   = Xoshiro256p(UInt64[0xdead, 0xbeef, 0xcafe, 0xbabe])
```

Seeds are validated by `checkseed`, or `checkseed63` for MRG63k3a: a valid seed of either has 6 non-negative values; the first three must be < m1 and not all zero, the last three < m2 and not all zero.

The standard Julia interface also works with every generator:

```
Random.seed!(rng, 42)           # integer seed (splitmix64 expansion)
Random.seed!(rng, [7,7,7,8,8,8]) # explicit MRG32k3a seed (validated)
```

2.7 Full Random API

RandomDataStreams generators are drop-in substitutes for Julia's built-in RNGs: anywhere a function accepts an AbstractRNG, you can pass an RandomDataStreams generator and keep your stream/substream control.

```
using RandomDataStreams, Random

rng = next_stream!(MRG32k3aGen()) # any RandomDataStreams generator works here

rand(rng, 5)                       # Vector{Float64}, 5 draws
A = rand(rng, Float64, 2, 3)       # 2x3 matrix
z = randn(rng)                     # standard normal
e = randexp(rng)                   # exponential
```

```
v = shuffle(rng, collect(1:8))      # shuffled copy
p = randperm(rng, 6)               # random permutation
buf = Vector{Float64}(undef, 3)
rand!(rng, buf)                   # fill in place
Random.randstring(rng, 10)         # random string
```

Seeding through the standard interface makes results reproducible:

```
Random.seed!(rng, 42)
u1 = [rand(rng) for _ in 1:3]
Random.seed!(rng, 42)
u2 = [rand(rng) for _ in 1:3]
u1 == u2                          # true
```

Only `sample` requires `StatsBase.jl` and is out of scope.

2.8 Next steps

- [Streams & substreams](#)
- [API reference](#)
- [Implementation notes](#)

Part III

Streams & Substreams

Chapter 3

Streams & Substreams

The central abstraction of RandomDataStreams.jl is the *stream*: a long, non-overlapping subsequence of a generator's period. Streams can themselves be split into *substreams*. This is the architecture recommended by L'Ecuyer et al. (2002) for parallel and replicated stochastic simulation:

- **Streams** are handed to independent replications or parallel workers; their outputs never overlap, so replications are statistically independent.
- **Substreams** delimit scenarios *within* one replication; rewinding a substream while keeping the scenario structure enables techniques such as common random numbers.

3.1 Why streams? The case for Common Random Numbers (CRN)

The whole point of the stream/substream architecture is **variance reduction when comparing systems** — arguably the most important technique in stochastic simulation design (L'Ecuyer et al. 2002; Law, *Simulation Modeling and Analysis*).

Suppose you compare K configurations of a stochastic system (two inventory levels, two queue disciplines...). The naive approach runs each configuration with *independent* random numbers. The variance of the estimated difference is then

$$\text{Var}[\hat{D}] = \text{Var}[\tilde{Y}_1]/n + \text{Var}[\tilde{Y}_2]/n$$

With **common random numbers**, every configuration of a given replication consumes the *same* underlying uniforms: the per-replication differences are strongly positively correlated and

$$\text{Var}[\hat{D}_{\text{CRN}}] = \text{Var}[\tilde{Y}_1]/n + \text{Var}[\tilde{Y}_2]/n - 2 \cdot \text{Cov}[\tilde{Y}_1, \tilde{Y}_2]/n$$

— often orders of magnitude smaller. But CRN only works if the random number usage can be *synchronised exactly* across configurations, replication after replication. That is precisely what RandomDataStreams's machinery guarantees:

Measurable example: comparing two inventory levels

Daily cost of an inventory system over 30 days (demands $N(100, 20)$ truncated at 0), comparing stock level $s = 50$ vs $s = 60$:

Need	RandomDataStreams tool
Replications must be independent	a fresh stream per replication (<code>next_stream!</code>)
Configurations must see identical randomness	rewind each configuration to the same substream start (<code>reset_stream!</code> / <code>reset_substream!</code>)
No accidental overlap between replications	streams are provably disjoint

```

using RandomDataStreams, Statistics

function cost(s, rng)
    total, stock = 0.0, Float64(s)
    for _ in 1:30
        stock = min(stock + 40, 200)
        d = max(0.0, 100 + 20randn(rng))
        sold = min(stock, d)
        total += 5(d - sold) + 0.1max(0, stock - sold - 10)
        stock -= sold
    end
    total
end

function replications(crn::Bool, n::Int)
    gen = MRG32k3aGen()
    diffs = Vector{Float64}(undef, n)
    for i in 1:n
        r50 = next_stream!(gen)
        l50 = cost(50, r50)
        r60 = crn ? reset_stream!(copy(r50)) : next_stream!(gen)
        l60 = cost(60, r60)
        diffs[i] = l60 - l50
    end
    return diffs
end

for crn in (true, false)
    ds = replications(crn, 4_000)
    println("CRN = \$crn: mean diff ≈ \$(round(mean(ds), digits=1)), ",
           "var(diff) ≈ \$(round(var(ds), digits=1))")
end

```

Typical output:

```

CRN = true:  mean diff ≈ -49.9, var(diff) ≈ 0.1
CRN = false: mean diff ≈ -39.2, var(diff) ≈ 540775.9

```

The variance of the comparison drops by **six orders of magnitude**: with CRN, both policies face identical demand sequences, so the observed difference measures the effect of the policy change rather than the noise of unrelated demand scenarios. Without CRN you would need $\sim 10^6$ more replications for the same precision.

This example uses `copy(r50) + reset_stream!`: both policies replay the very same stream from its beginning. `reset_substream!` generalises the pattern when each replication itself contains several scenarios.

3.2 The two object families

Every generator in the package comes as a pair, and the distinction is the one the whole interface rests on.

1. **Generator objects** (<: AbstractRNGStream) are factories. They hold the seed of the *next* stream and hand out streams with `next_stream!`. They produce no random numbers themselves.
2. **Streams** (<: AbstractStreamableRNG) produce the numbers, and move along the stream and sub-stream hierarchy. They are AbstractRNGs, so they work anywhere Julia's own generators do.

Family	Generator object	Stream
MRG32k3a	MRG32k3aGen	MRG32k3a
MRG63k3a	MRG63k3aGen	MRG63k3a
xoroshiro128	Xoroshiro128pGen, Xoroshiro128ssGen, Xoroshiro128ppGen	Xoroshiro128p, Xoroshiro128ss, Xoroshiro128pp
xoshiro256	Xoshiro256pGen, Xoshiro256ssGen, Xoshiro256ppGen	Xoshiro256p, Xoshiro256ss, Xoshiro256pp
xoshiro512	Xoshiro512pGen, Xoshiro512ssGen, Xoshiro512ppGen	Xoshiro512p, Xoshiro512ss, Xoshiro512pp
PCG	PCG64Gen, PCG64DXSMGen	PCG64, PCG64DXSM
Philox	PhiloxGen (4x32-10), Philox4x64Gen	PhiloxRNG, Philox4x64RNG
Threefry	Threefry4x32Gen, Threefry4x64Gen	Threefry4x32RNG, Threefry4x64RNG

Xoshiro256plusGen is an alias of Xoshiro256pGen, kept from earlier versions.

The interface is the same across the table: whatever the family, a generator object answers to `next_stream!`, `srand!`, `get_state` and `set_state!`, and a stream answers to `next_substream!`, `reset_substream!`, `reset_stream!`, `advance_state!`, `get_state`, `set_state!` and `srand!`. Code written against the two abstract types never names a family — see the [API reference](#).

3.3 Producing independent streams

```
using RandomDataStreams

gen = MRG32k3aGen()           # or Xoshiro256plusGen(UInt64[...])

worker1 = next_stream!(gen)    # stream 1
worker2 = next_stream!(gen)    # stream 2 – guaranteed disjoint from stream 1
worker3 = next_stream!(gen)    # stream 3 – ...
```

Each call to `next_stream!` advances the generator's internal seed by a huge leap (2^{127} values for MRG32k3a, 2^{250} for MRG63k3a, a full `long_jump!` for Xoshiro256+), which is what guarantees non-overlap.

3.4 Threads

Streams share no state, so one stream per thread needs no synchronisation. Take the streams first, then parallelise over them:

```
using RandomDataStreams, Base.Threads
```

```

gen = MRG32k3aGen()           # any generator object
rngs = next_stream!(gen, nthreads()) # n streams, taken serially

results = Vector{Float64}(undef, nthreads())
@threads for t in 1:nthreads()
    rng = rngs[t]              # this thread owns this stream
    results[t] = sum(rand(rng) for _ in 1:10^5)
end

```

The results are those of the same streams drawn one after another, which the test suite checks for every family.

Do not call 'next_stream!' inside the parallel loop

A generator object holds the seed of the next stream and rewrites it on every call. Concurrent calls read the same seed, so the streams handed out overlap — and nothing reports it. In a measurement here, 64 threads calling `next_stream!` on one shared generator received between 57 and 63 distinct streams instead of 64, on three consecutive runs. The failure is silent and destroys precisely the guarantee this package exists to provide.

`next_stream!(gen, n)` exists so that the correct pattern is also the shorter one.

3.5 Distributed

Separate processes cannot share stream objects, so the seeds travel instead. Take them from the generator before each `next_stream!`, and rebuild the stream on the worker with `srand!`:

```

using Distributed
@everywhere using RandomDataStreams

gen = MRG32k3aGen()
seeds = Vector{Any}(undef, nworkers())
for i in 1:nworkers()
    seeds[i] = get_state(gen)      # seed of the stream about to be produced
    next_stream!(gen)             # advance past it
end

@sync for (i, w) in enumerate(workers())
    seed = seeds[i]
    @spawnat w begin
        rng = MRG32k3a(1)         # any valid seed; replaced on the next line
        srand!(rng, seed)         # this worker's stream, from its beginning
        # ... draw from rng ...
    end
end

```

Use `srand!` here, not `set_state!`. They are not interchangeable: `set_state!` moves the current position only, leaving the stream and substream boundaries where the constructor put them, so a later `reset_stream!` on the worker would rewind to the wrong place. `srand!` sets all three checkpoints, which is what makes the worker's stream a stream rather than a position in someone else's.

Both are portable across families, as is `get_state` on a generator object; the representation that travels is opaque, so hand it back unchanged rather than interpreting it.

3.6 Navigating substreams

Within a single RNG, three functions move along the stream/substream hierarchy (all return the modified RNG):

Function	Effect
<code>reset_stream!(rng)</code>	jump to the very beginning of the current stream
<code>reset_substream!(rng)</code>	jump to the beginning of the current substream
<code>next_substream!(rng)</code>	advance to the next substream

Example — common random numbers across two scenarios:

```
rng = next_stream!(MRG32k3aGen())

rand(rng)           # consume some variates of substream 1...
rand(rng)

next_substream!(rng) # switch to substream 2 for scenario B
uB = rand(rng)

reset_stream!(rng)   # replay substream 1 from scratch for scenario A
uA = rand(rng)       # same underlying uniforms as the first draw
```

For Xoshiro256+, `next_substream!` is implemented as a `short_jump!` (2^{96} -value leap) and `reset_stream!/next_substream!` manipulate the saved Bg/Ig checkpoints.

3.7 Saving, restoring, jumping

```
state = get_state(rng) # a copy; safe to store
# ... consume numbers ...
```

To restore an MRG32k3a or MRG63k3a state, rebuild the generator with the triple constructor (Cg, Bg, Ig all set to the snapshot):

```
snapshot = get_state(rng)
xs = [rand(rng) for _ in 1:5]
clone = MRG32k3a(snapshot, snapshot, snapshot)
rand(clone) == xs[1] # true - resumes exactly after the snapshot
```

Both MRG generators additionally support arbitrary jumps inside a stream:

```
advance_state!(rng, e, c)
```

moves the state forward by n steps, where $n = 2^e + c$ (e may be negative, and negative c moves backwards). This costs $O(\log n)$ matrix operations, independent of the distance jumped.

3.8 Scope: host-side streams, device-side bijections

The stream object of L'Ecuyer et al. (2002) is stateful by construction — a current position, a substream anchor, a stream anchor, mutated in place. That makes it a **host-side** abstraction. A GPU kernel wants the opposite: no state at all, one value computed from the thread index. The package does not run on a device, and that is a scope boundary rather than a gap, because the two halves fit together.

Counter-based generators: the addressing scheme is the work assignment. The key names the stream, the high half of the counter the substream, the low half the position. Any draw can therefore be recomputed from (key, substream, index) alone, with no object, which is exactly what a kernel needs:

```
using RandomDataStreams
const RDS = RandomDataStreams

function direct_word(key::NTuple{2, UInt32}, substream::Integer, i::Integer)
    block, word = divrem(i, 4) # four 32-bit words per block
    ctr = (UInt128(substream) << 64) | UInt128(block)
    c = (ctr % UInt32, (ctr >> 32) % UInt32, (ctr >> 64) % UInt32, (ctr >> 96) % UInt32)
    return RDS.philox(c, key)[word + 1]
end
```

This agrees with the stream object draw for draw; the test suite checks it, so host and device address the same sequence. philox and threefry are pure, allocation-free functions of their arguments, which is the necessary condition for using them inside a kernel — necessary, not sufficient: the package has no GPU dependency and runs no device tests, so that last step is the user's.

Recurrence-based generators: the host computes the starting points. There is no stateless form here, and the standard pattern runs the other way: use next_stream! on the host to produce as many non-overlapping starting states as there are tasks, ship one to each, and let each iterate its own recurrence.

```
gen = Xoshiro256plusGen(UInt64[1, 2, 3, 4])
seeds = [get_state(next_stream!(gen)) for _ in 1:nworkers] # non-overlapping
```

The jump machinery is what makes this cheap — matrix jumps for MRG32k3a, GF(2) polynomial jumps for the xoshiro families — and it is the construction L'Ecuyer et al. (2021, Sec. 2) describe for parallel environments.

What the package does not provide: filling a CuArray, or any device-side rand. If that is what you need, take the bijections and the addressing scheme above, and keep the stream objects on the host for what they are good at — assigning non-overlapping work and replaying it identically.

3.9 One interface, every generator

The whole point of the two object families is that code written against them does not name a generator. Every family in the package answers to the same calls, with the same meanings — the table below is asserted for all seventeen generators in the test suite, not just documented.

On the generator object:

On the stream:

The seeding rule. A value in the family's own representation — its seed vector, or a UInt128 for PCG — is the state or key. Any other integer is a *seed*, expanded through splitmix64 and folded into whatever the family accepts. So MRG32k3aGen(12345), Xoshiro256ppGen(12345), PhiloxGen(12345) and PCG64Gen(12345) all mean the same kind of thing, and T(12345) is equivalent to Random.seed!(T(), 12345) everywhere.

call	meaning
<code>Gen()</code>	seeded with the package default, 12345
<code>Gen(12345)</code>	seeded with an integer
<code>Gen(v)</code>	seeded with the family's own seed vector
<code>next_stream!(gen)</code>	the next non-overlapping stream
<code>srand!(gen, seed)</code>	reset the seed the next <code>next_stream!</code> will use
<code>get_state(gen), set_state!(gen, s)</code>	save and restore it

call	meaning
<code>T(), T(12345), T(v)</code>	the same three seeding forms
<code>next_substream!, reset_substream!, reset_stream!</code>	navigation; each returns the generator
<code>advance_state!(rng, e, c)</code>	move by $2^e + c$ draws, negative allowed
<code>get_state, set_state!</code>	save and restore the position only
<code>srand!, Random.seed!</code>	reseed, resetting both boundaries
<code>copy, show, the full Random API</code>	as for any <code>AbstractRNG</code>

What is not portable. `short_jump!` and `long_jump!` are the xoshiro and PCG spelling of a jump by exactly the substream and stream distance. Use `next_substream!` in code meant to work with any generator. `long_jump!` has no meaning at all for a counter-based generator, where moving to another stream means taking another key — an operation that belongs to the generator object.

3.10 Guarantees

- Streams produced by successive calls to `next_stream!` on the same generator object are **provably non-overlapping**.
- Substreams likewise partition a stream into disjoint blocks (length $\approx 2^{76}$ steps for MRG32k3a, 2^{150} for MRG63k3a).

Part IV

API Reference

Chapter 4

API Reference

All names below are exported by the `RandomDataStreams` module unless marked (*internal*).

4.1 Types

AbstractRNGStream — supertype of stream *generators*: objects that mint new independent streams. MRG32k3aGen, MRG63k3aGen, the nine Xoshiro*Gen/Xoroshiro*Gen, PCG64Gen, PCG64DXSMGen, PhiloxGen, Philox4x64Gen, Threefry4x32Gen, Threefry4x64Gen.

AbstractStreamableRNG <: Random.AbstractRNG — supertype of usable RNGs that navigate streams and substreams. MRG32k3a, MRG63k3a, the nine xoshiro/xoroshiro variants, PCG64, PCG64DXSM, PhiloxRNG, Philox4x64RNG, Threefry4x32RNG, Threefry4x64RNG.

4.2 The common interface

Everything in this section works on **every** generator the package ships, with the same meaning. It is asserted for all seventeen in the test suite, so code written against `AbstractStreamableRNG` never has to name a family. The per-family sections below add only what is specific.

Seeding

Form	Meaning
<code>T(), Gen()</code>	the package default seed, 12345
<code>T(12345), Gen(12345)</code>	an integer seed , expanded through <code>splitmix64</code>
<code>T(v), Gen(v)</code>	the family's own representation — its seed vector, or a <code>UInt128</code> for PCG — taken as the state or key itself
<code>Random.seed!(rng, 12345)</code>	same as <code>T(12345)</code> , in place
<code>srand!(rng, seed),</code> <code>srand!(gen, seed)</code>	reseed, resetting both boundaries

The distinction is the whole rule: an integer is a *seed* and gets hashed; a value shaped like the state *is* the state.

On the stream

`get_state` returns a family-specific representation. Treat it as opaque and only ever hand it back to `set_state!`.

Call	Meaning
<code>next_substream!(rng) -> rng</code>	move to the start of the next substream
<code>reset_substream!(rng) -> rng</code>	rewind to the start of the current substream
<code>reset_stream!(rng) -> rng</code>	rewind to the start of the current stream
<code>advance_state!(rng, e, c) -> rng</code>	move by $2^e + c$ draws; negative moves backwards
<code>get_state(rng) / set_state!(rng, s) -> rng</code>	save and restore the position only
<code>copy(rng), show(io, rng)</code>	independent copy; one-line summary, or the three anchors when displayed at the REPL
the full Random API	<code>rand, rand!, randn, shuffle, ranges</code> , every integer and float type

On the generator object

Call	Meaning
<code>next_stream!(gen) -> rng</code>	the next non-overlapping stream
<code>next_stream!(gen, n) -> Vector{rng}</code>	the next n of them; the thread-safe way to obtain streams for parallel work
<code>srand!(gen, seed) -> gen</code>	reset the seed the next <code>next_stream!</code> will use
<code>get_state(gen) / set_state!(gen, s) -> gen</code>	save and restore that seed
<code>show(io, gen)</code>	display it, on one line or in full as above

What is *not* common

Name	Where	Why not everywhere
<code>short_jump!(rng)</code>	xoshiro/xoroshiro, PCG	the family's spelling of a jump by exactly the substream distance; use <code>next_substream!</code> in portable code
<code>long_jump!(rng)</code>	xoshiro/xoroshiro, PCG	jumps by the stream distance in place. Meaningless for a counter-based generator, where changing stream means changing key, which belongs to the generator object
<code>checkseed</code> , <code>checkseed63</code>	MRG32k3a and MRG63k3a, xoshiro/xoroshiro	those two families have invalid seeds — the MRG moduli with no all-zero component, and the xoshiro all-zero state, which is a fixed point. PCG and the counter-based families accept every value
<code>stream_key</code>	counter-based	the key schedule hook; there is nothing to schedule in a recurrence-based generator
<code>next(rng)</code> (<i>internal</i>)	all	raw word output, before any conversion

4.3 MRG32k3a

Constructors

Seeds are validated with `checkseed`; invalid seeds throw an `AssertionError`.

Signature	Description
<code>MRG32k3a()</code>	default seed [12345, 12345, 12345, 12345, 12345, 12345]
<code>MRG32k3a(seed::Integer)</code>	integer seed, folded into the valid seed space
<code>MRG32k3a(x::Vector{Int})</code>	seed all three states ($C_g = B_g = I_g = x$)
<code>MRG32k3a(x, y, z::Vector{Int})</code>	explicit current/substream/stream starts

Functions

rand(rng::MRG32k3a) -> Float64 Uniform in $[0, 1)$ with ~ 32 bits of resolution. This is the native output.

rand(rng::MRG32k3a, T) Supported T: Float64, Float32, Float16, UInt8...UInt128, Int8...Int128, Bool, and any UnitRange (e.g. `rand(rng, 1:10)`).

reset_stream!(rng) -> rng Set the current state (C_g) and the substream start (B_g) back to the stream start (I_g).

reset_substream!(rng) -> rng Set C_g back to B_g .

next_substream!(rng) -> rng Advance B_g to the next substream boundary (jump of $\approx 2^{76}$ steps) and set $C_g = B_g$.

advance_state!(rng, e::Integer, c::Integer) -> rng Jump forward/backward inside the current substream by n steps where $n = 2^e + c$. Negative e or c move backwards. Cost is $O(\max(|e|, \log |c|))$ matrix operations.

get_state(rng) -> Vector{Int} Return a copy of C_g .

checkseed(x) -> Bool true iff x has length 6, all entries ≥ 0 , entries 1–3 are $< m_1$ and not all zero, entries 4–6 are $< m_2$ and not all zero.

DEFAULT_SEED The constant seed vector used by `MRG32k3a()` and `MRG32k3aGen()`.

copy(m::MRG32k3a) -> MRG32k3a Independent deep copy of the generator (all three state vectors).

Stream generation

Signature	Description
<code>MRG32k3aGen()</code> , <code>MRG32k3aGen(12345)</code> , <code>MRG32k3aGen(v)</code>	create a stream generator
<code>next_stream!(gen) -> MRG32k3a</code>	return a new stream; internal seed leaps 2^{127} values ahead
<code>srand!(gen, seed) -> gen</code>	reset the stored seed
<code>get_state(gen) -> Vector{Int}</code> / <code>set_state!(gen, s)</code>	save and restore it

4.4 MRG63k3a

The same combined MRG in 64-bit arithmetic: L'Ecuyer (1999), Table II, fourth entry. Two moduli just under 2^{63} ($m_1 = 2^{63} - 6645$, $m_2 = 2^{63} - 21129$), period $\approx 2^{377}$, and just under 63 random bits per step against 32 for MRG32k3a. Everything below mirrors the MRG32k3a section; only the numbers differ.

Signature	Description
<code>MRG63k3a()</code>	default seed [12345, 12345, 12345, 12345, 12345]
<code>MRG63k3a(seed::Integer)</code>	integer seed, folded into the valid seed space
<code>MRG63k3a(x::Vector{Int})</code>	seed all three states ($C_g = B_g = I_g = x$)
<code>MRG63k3a(x, y, z::Vector{Int})</code>	explicit current/substream/stream starts

Constructors

Seeds are validated with `checkseed63`; invalid seeds throw an `AssertionError`.

Functions

rand(rng::MRG63k3a) -> Float64 Uniform in (0, 1), from an integer with ~63 bits of resolution, of which a Float64 keeps 53. This is the native output, and it reproduces L'Ecuyer's C implementation value for value.

rand(rng::MRG63k3a, T) Supported T: as for MRG32k3a. Words are assembled from 32-bit chunks, so a UInt64 costs two steps (four for MRG32k3a) and a UInt32 one.

next_substream!(rng) -> rng Advance B_g by 2^{150} steps and set $C_g = B_g$.

reset_stream!, reset_substream!, advance_state!, get_state, set_state!, srand!, copy Exactly as for MRG32k3a, with one internal difference that does not show: MRG63k3a keeps its state one step ahead of the position (Vigna's third optimization, worth 7% here), so `get_state` returns the seed representation by stepping the stored vector back, and the constructors, `set_state!` and `Random.seed!` step a seed forward. Reading `rng.Cg` directly is therefore not the same thing as `get_state(rng)` for this generator. See the [Implementation Notes](#).

checkseed63(x) -> Bool true iff x has length 6, all entries ≥ 0 , entries 1-3 are $< m_1$ and not all zero, entries 4-6 are $< m_2$ and not all zero.

DEFAULT_SEED63 The constant seed vector used by `MRG63k3a()` and `MRG63k3aGen()`.

Stream generation

Signature	Description
<code>MRG63k3aGen(), MRG63k3aGen(12345), MRG63k3aGen(v)</code>	create a stream generator
<code>next_stream!(gen) -> MRG63k3a</code>	return a new stream; internal seed leaps 2^{250} values ahead
<code>srand!(gen, seed) -> gen</code>	reset the stored seed
<code>get_state(gen) -> Vector{Int} / set_state!(gen, s)</code>	save and restore it

The jumps are unaffected by that shift: the jump matrices commute with the one-step matrix, so a jumped shifted state is the shifted jumped state.

The stream and substream distances are not L'Ecuyer's: he published jump matrices for MRG32k3a only. 2^{250} and 2^{150} are this package's choice, scaled from his 2^{127} and 2^{76} by the ratio of the two periods, and the matrices are computed at precompilation rather than tabulated.

4.5 Xoshiro256+

Constructors

Signature	Description
<code>Xoshiro256p()</code>	the package default seed, 12345, through splitmix64
<code>Xoshiro256p(seed::Integer)</code>	integer seed, through splitmix64 (never all-zero)
<code>Xoshiro256p(s::NTuple{4, UInt64}) /</code> <code>Xoshiro256p(v::Vector{<:Unsigned})</code>	$C_g = B_g = I_g = s$
<code>Xoshiro256p(x, y, z)</code>	explicit current/substream/stream states

Functions

rand(rng::Xoshiro256p) -> Float64 Uniform in [0, 1), built from a full 64-bit draw.

rand(rng::Xoshiro256p, T) Supported T: Float64, Float32, Float16, and the integer and Bool types of the Random API. Ranges (`rand(rng, 1:10)`) go through the standard Random sampler, which rejects rather than folding and is therefore unbiased.

srand!(rng, seed::Vector{UInt64}) -> rng Re-seed an existing generator (first 4 words are used).

reset_stream!(rng) -> rng, reset_substream!(rng) -> rng, next_substream!(rng) -> rng Same semantics as for MRG32k3a; `next_substream!` performs a `short_jump!` (leap of $\approx 2^{96}$ values).

short_jump!(rng) -> rng Jump to the beginning of the next 2^{96} -value block (Vigna's short jump).

long_jump!(rng) -> rng Leap $\approx 2^{192}$ values ahead — used to delimit streams.

get_state(rng) -> Vector{UInt64} Copy of the current state C_g .

copy(rng) -> Xoshiro256p Independent copy.

(internal) **next(rng) -> UInt64**: raw 64-bit output; also available as `rand(rng, UInt64)`.

Stream generation

Signature	Description
<code>Xoshiro256plusGen(), Xoshiro256plusGen(12345),</code> <code>Xoshiro256plusGen(v)</code>	create a stream generator (a seed <i>vector</i> must have exactly 4 words)
<code>next_stream!(gen) -> Xoshiro256p</code>	new independent stream (<code>long_jump!</code> from the stored seed)
<code>srand!(gen, seed::Vector{UInt64}) -> gen</code>	reset the stored seed
<code>get_state(gen) -> Vector{UInt64}</code>	seed that will be used by the next <code>next_stream!</code> call

4.6 Xoshiro / xoroshiro families

All variants share one implementation (`LinRNG{N,S}` internally, N = state words, S = scrambler) and expose the same API as [Xoshiro256+](#) above. Exported types:

RNGs	Stream generators
Xoroshiro128p, Xoroshiro128ss, Xoroshiro128pp	Xoroshiro128pGen, Xoroshiro128ssGen, Xoroshiro128ppGen
Xoshiro256p, Xoshiro256ss, Xoshiro256pp	Xoshiro256plusGen (also Xoshiro256pGen), Xoshiro256ssGen, Xoshiro256ppGen
Xoshiro512p, Xoshiro512ss, Xoshiro512pp	Xoshiro512pGen, Xoshiro512ssGen, Xoshiro512ppGen

Constructors accept an `NTuple{N, UInt64}` or a `Vector{<:Unsigned}` of length `N` (1 or 3 arguments). Available functions: `rand` (`Float64/Float32/Float16`, `UInt64`, ranges), `srand!`, `reset_stream!`, `reset_substream!`, `next_substream!`, `short_jump!`, `long_jump!`, `get_state`, `copy`.

advance_state!(rng, e::Integer, c::Integer) -> rng Jump forward by `n` steps ($n = 2^e + c$ if $e > 0$, $n = -2^(-e) + c$ if $e < 0$, $n = c$ if $e = 0$) or backward if $n < 0$. Implemented by computing the jump polynomial $x^n \bmod p(x)$ over $\text{GF}(2)$ (p = the family's characteristic polynomial) and applying it to the current state; distances are reduced modulo the period. Only Cg moves — stream/substream boundaries are kept. Cost is $O((64N)^2)$ $\text{GF}(2)$ operations (≈ 10 –500 ms depending on family); for the standard distances prefer `short_jump! / long_jump!`, which use precomputed constants and are allocation-free.

Jump semantics (all xoshiro-family generators):

- `short_jump!(rng)` — jump anchored at the current **substream start** (Bg); lands at the next substream boundary (2^{64} values for xoroshiro128, 2^{128} for xoshiro256, 2^{256} for xoshiro512).
- `long_jump!(rng)` — jump anchored at the **stream start** (Ig); delimits independent streams.
- `next_stream!(gen)` applies the long-jump polynomial to the stored seed.

All jump constants are the official ones from xoshiro.di.unimi.it and outputs are validated byte-for-byte against the original C implementations.

4.7 Where the families actually differ

Every call in [The common interface](#) is available on all seventeen generators, so there is no availability matrix to consult. What differs is cost and representation:

For the standard distances on a xoshiro or PCG generator prefer `short_jump! / long_jump!` over `advance_state!`: they use precomputed constants (xoshiro) or a single doubling chain (PCG) and are allocation-free.

4.8 PCG

PCG (O'Neill 2014) runs a 128-bit linear congruential recurrence and permutes the state into 64 bits of output. PCG64 is the default bit generator of NumPy, and the outputs here match it exactly for the same state; PCG64DXSM is the variant NumPy recommends for large-scale parallel work.

Streams come from the closed-form LCG jump, not from PCG's own increment-based stream mechanism, which this package deliberately does not expose — see the [Implementation Notes](#) for the reason and the [FAQ](#) for what to do instead when reproducing a NumPy pipeline.

`RandomDataStreams.PCG64` – Type.

	MRG32k3a	MRG63k3a	xoshiro / xoroshiro	PCG64, PCG64DXSM	Philox, Threefry
Native output	Float64, ~32-bit resolution	Float64, ~63-bit resolution	UInt64	UInt64	a block of four words
get_state returns	Vector{Int} (6)	Vector{Int} (6)	Vector{UInt64} (N)	UInt128	(ctr, key, buffer, idx)
Cost of advance_state!	O(e) matrix products	O(e) matrix products	O((64N) ²) GF(2) ops, ~10-500 ms	O(log n) multiplies	O(1), a counter addition
short_jump!, long_jump!	—	—	yes	yes	—
Rejected seeds	checkseed: two moduli, no all-zero component	checkseed63: same, wider moduli	checkseed: the all-zero state	none	none
Whole-block rand!	—	—	—	—	yes, ~1.7× the scalar path
Extra hooks	checkseed, DEFAULT_SEED	checkseed63, DEFAULT_SEED63	jump polynomials	—	stream_key

PCG64 <: AbstractStreamableRNG

PCG64 (O'Neill 2014) with the XSL-RR output permutation: a 128-bit linear congruential state, period 2^{128} , 64 bits of output per draw. This is the default bit generator of NumPy, and the outputs here match it exactly for the same state.

Streams are 2^{96} values apart and substreams 2^{64} , giving 2^{32} streams of 2^{32} substreams — the same scale as Xoroshiro128p, so it suits mild parallelism rather than large simulations. Jumping to an arbitrary position costs $O(\log n)$ multiplications, the cheapest of any generator in the package.

The increment is fixed: PCG's own increment-based "streams" are not used here, because two increments can define orbits that differ by a constant offset. See the implementation notes.

[source](#)

RandomDataStreams.PCG64DXSM – Type.

PCG64DXSM <: AbstractStreamableRNG

PCG64 with the DXSM output permutation and the cheap 64-bit multiplier, the variant NumPy recommends for large-scale parallel use. Same state size, period and stream structure as [PCG64](#), and the outputs match NumPy's PCG64DXSM exactly for the same state.

[source](#)

RandomDataStreams.PCG64Gen – Type.

```
PCG64Gen([seed])
```

Hands out independent [PCG64](#) streams, $2^{32} * (2^{64} + 1) + 1$ values apart.

[source](#)

`RandomDataStreams.PCG64DXSMGen` – Type.

```
PCG64DXSMGen([seed])
```

Hands out independent [PCG64DXSM](#) streams, $2^{32} * (2^{64} + 1) + 1$ values apart.

[source](#)

`RandomDataStreams.PCGRNG` – Type.

```
PCGRNG{V}
```

Permuted congruential generator on 128 bits of state, where *V* names the output permutation. Use the aliases [PCG64](#) and [PCG64DXSM](#).

Like the other recurrence-based generators of the package it carries three checkpoints: the current state *Cg*, the start of the current substream *Bg* and the start of the current stream *Ig*.

[source](#)

`RandomDataStreams.PCGGen` – Type.

```
PCGGen{V} <: AbstractRNGStream
```

Hands out non-overlapping [PCGRNG](#) streams: each call to `next_stream!` applies the long jump to the stored seed. Use the aliases [PCG64Gen](#) and [PCG64DXSMGen](#).

[source](#)

4.9 Counter-based generators

A counter-based RNG produces its output block as a keyed bijection of a counter, $R = b_K(C)$, rather than by iterating a state transition. CBRNG holds the machinery every such family shares — counter, key, block buffer, streams and substreams — so a variant only supplies its bijection.

`RandomDataStreams.CBRNG` – Type.

```
CBRNG{B,W,N,K}
```

Counter-based generator, where *B* names the bijection, *W* is the word type, *N* the number of words in an output block and *K* the number of key words. Concrete aliases ([PhiloxRNG](#), [Philox4x64RNG](#), ...) are what user code normally names.

Streams are indexed by the key, substreams by the high half of the counter; see [next_stream!](#) and [next_substream!](#).

[source](#)

`RandomDataStreams.stream_key` – Function.

```
stream_key(::Val{B}, seed) -> key
```

Key schedule of the family *B*: the bijection mapping the seed of a stream — the position 0, 1, 2, ... walked by [CBGen](#), or a value the user supplied through `srand!` — to the key the bijection is actually keyed with.

The default is the identity, which is what [Philox](#) uses: L'Ecuyer et al. (2021, Sec. 3) report that consecutive small keys are safe there, on the strength of the statistical testing of [Salmon et al. \(2011\)](#) and the ten rounds of encoding.

A family whose bijection is weak for structured keys **must** override this method with a schedule that hashes the seed. The paper's worked example is Squares (Widynski 2020): key 0 makes the output identically zero, and a small key k makes roughly the first $2^{16} / k$ outputs zero, so handing out the seeds 1, 2, ..., s as keys "will be very bad unless the keys are hashed to random-looking values by another mechanism".

[source](#)

Philox

`RandomDataStreams.PhiloxGen` – Type.

```
PhiloxGen([key])
```

Hands out independent `PhiloxRNG` streams, one distinct key each.

[source](#)

`RandomDataStreams.PhiloxRNG` – Type.

```
PhiloxRNG([key, [ctr]])
```

Philox4x32-10 counter-based stream. The key identifies the stream, the 128-bit counter its position; sub-streams are 2^{64} blocks (2^{66} draws) apart.

[source](#)

`RandomDataStreams.Philox4x64Gen` – Type.

```
Philox4x64Gen([key])
```

Hands out independent `Philox4x64RNG` streams, one distinct key each.

[source](#)

`RandomDataStreams.Philox4x64RNG` – Type.

```
Philox4x64RNG([key, [ctr]])
```

Philox4x64-10 counter-based stream: same construction as `PhiloxRNG` with 64-bit words, so a block holds four `UInt64` and `rand(rng, UInt64)` costs no bit assembly. Only the low 128 bits of its 256-bit counter space are used.

[source](#)

Threefry

Same construction as Philox, with no multiplication at all: every round is add / rotate / xor, which makes Threefry reproducible bit-for-bit across architectures, including ones with no fast integer multiply. Salmon et al. (2011) found it the fastest of the family on the CPUs of the time and recommend twenty rounds there; on a current x86 our own measurement puts Philox4x64-10 ahead (see the performance notes).

`RandomDataStreams.Threefry4x64Gen` – Type.

```
Threefry4x64Gen([seed])
```

Hands out independent `Threefry4x64RNG` streams, one distinct key each.

[source](#)

`RandomDataStreams.Threefry4x64RNG` - Type.

```
Threefry4x64RNG([key, [ctr]])
```

Threefry4x64-20 counter-based stream, the variant Salmon et al. recommend on CPUs. Four 64-bit key words identify the stream; substreams are 2^{64} blocks apart. Only the low 128 bits of its 256-bit counter space are used, which still gives 2^{130} draws per substream.

The round count is a parameter of the bijection rather than of the type, so a faster 13-round variant — enough to pass Crush, per Salmon et al. — is three lines:

```
RandomDataStreams.bijection(:Val{:threefry4x64_13}, ctr:NTuple{4,UInt64},
↪ key:NTuple{4,UInt64}) =
  RandomDataStreams.threefry(ctr, key, Val(13))
RandomDataStreams._variant_name(:Type{<:RandomDataStreams.CBRNG{:threefry4x64_13}}) =
↪ "Threefry4x64-13"
const Threefry4x64_13RNG = RandomDataStreams.CBRNG{:threefry4x64_13,UInt64,4,4}
```

[source](#)

`RandomDataStreams.Threefry4x32Gen` - Type.

```
Threefry4x32Gen([seed])
```

Hands out independent `Threefry4x32RNG` streams, one distinct key each.

[source](#)

`RandomDataStreams.Threefry4x32RNG` - Type.

```
Threefry4x32RNG([key, [ctr]])
```

Threefry4x32-20 counter-based stream: same construction as `Threefry4x64RNG` with 32-bit words, for a 128-bit counter and a 128-bit key.

[source](#)

Part V

Docstrings

Chapter 5

Docstrings

Auto-generated documentation from the package docstrings.

5.1 Abstract types

`RandomDataStreams.AbstractRNGStream` – Type.

`AbstractRNGStream` is an abstraction for creating RNG objects with non-overlapping random numbers.

[source](#)

`RandomDataStreams.AbstractStreamableRNG` – Type.

`AbstractStreamableRNG` is a special kind of RNG where one can move between different stream of number that are known to be non-overlapping.

Every concrete subtype implements the same interface, so that code written against `AbstractStreamableRNG` works with any generator of the package:

operation	meaning
<code>next_substream!(rng)</code>	move to the start of the next substream
<code>reset_substream!(rng)</code>	rewind to the start of the current substream
<code>reset_stream!(rng)</code>	rewind to the start of the current stream
<code>advance_state!(rng, e, c)</code>	move by n draws, n given by (e, c); n < 0 moves backwards
<code>get_state(rng)</code>	current position, in a generator-specific representation
<code>set_state!(rng, state)</code>	restore a position obtained from <code>get_state</code>
<code>srand!(rng, seed)</code>	reseed, resetting the stream and substream boundaries

The unit of `advance_state!` is one `rand(rng)` draw, for every generator, so that a jump written against `AbstractStreamableRNG` means the same thing everywhere. `get_state` and `set_state!` are inverses — `set_state!(rng, get_state(rng))` leaves `rng` unchanged — and they move only the current position, never the stream or substream boundaries. The representation returned by `get_state` differs between families, so generic code should treat it as opaque and only ever hand it back to `set_state!`.

Streams themselves are produced by an `AbstractRNGStream` object, which implements the same operations on its side:

An integer seed is expanded through `splitmix64` and folded into whatever the family accepts, so `MRG32k3aGen(12345)`, `Xoshiro256ppGen(12345)`, `PhiloxGen(12345)` and `PCG64Gen(12345)` all mean the same kind of thing.

operation	meaning
<code>Gen()</code>	a generator seeded with the package default, 12345
<code>Gen(seed)</code>	seeded with an integer, or with the family's own seed vector
<code>next_stream!(gen)</code>	the next non-overlapping stream
<code>next_stream!(gen, n)</code>	the next n of them, as a vector
<code>srand!(gen, seed)</code>	reset the seed the next <code>next_stream!</code> will use
<code>get_state(gen) / set_state!(gen, s)</code>	save and restore that seed

A generator object is a factory, and it is **not thread-safe**: it rewrites the seed of the next stream on every call, so concurrent calls to `next_stream!` hand out overlapping streams, silently. Take the streams first, with `next_stream!(gen, n)`, then parallelise over them; streams themselves share no state and need no synchronisation.

Not part of this contract: `short_jump!` and `long_jump!`, which are the xoshiro and PCG spelling of a jump by the substream and stream distance. Use `next_substream!` for portable code. `long_jump!` has no meaning for a counter-based generator at all, where moving to another stream means taking another key, an operation that belongs to the generator object.

[source](#)

5.2 MRG32k3a

`RandomDataStreams.MRG32k3a` – Type.

`MRG32k3a()` will generate an instance of `MRG32k3a` with initial seeds `DEFAULT_SEED = [ 12345, 12345, 12345, 12345, 12345, 12345 ]`

`MRG32k3a(x::Vector{Int})` will generate an instance of `MRG32k3a` with initial seeds `x`

`MRG32k3a(x::Vector{Int}, y::Vector{Int}, z::Vector{Int})` will generate an instance of `MRG32k3a` with `Cg = x`, `Bg = y` and `Ig = z`

Examples

```
julia> rng = MRG32k3a();
```

```
julia> rand(rng)
0.12701112204657714
```

[source](#)

`RandomDataStreams.MRG32k3aGen` – Type.

An object that generates independent random number streams.

[source](#)

`RandomDataStreams.checkseed` – Function.

```
checkseed(s::NTuple{N,UInt64}) -> Bool
checkseed(v::AbstractVector{<:Unsigned}) -> Bool
```

true unless the words are all zero. The all-zero state maps to itself under every xoshiro/xoroshiro transition, so it is the one state the families forbid; there is no other constraint. (The `Vector{Int}` method is `MRG32k3a`'s, whose seed space is constrained differently.)

[source](#)

5.3 MRG63k3a

`RandomDataStreams.MRG63k3a` – Type.

`MRG63k3a()` will generate an instance of `MRG63k3a` with initial seeds `DEFAULT_SEED63 = [ 12345, 12345, 12345, 12345, 12345, 12345, 12345, 12345 ]`

`MRG63k3a(x::Vector{Int})` will generate an instance of `MRG63k3a` with initial seeds `x`

`MRG63k3a(x::Vector{Int}, y::Vector{Int}, z::Vector{Int})` will generate an instance of `MRG63k3a` positioned at `x`, with the substream starting at `y` and the stream at `z`. All three are seeds, in the representation `get_state` returns; the state actually stored runs one step ahead of them (see below).

`MRG63k3a` is the 64-bit-arithmetic member of L'Ecuyer's (1999) combined MRG family: same structure as `MRG32k3a`, moduli just under 2^{63} instead of 2^{32} , period $\approx 2^{377}$ instead of 2^{191} . Its step yields just under 63 random bits, against just under 32 for `MRG32k3a`, so a machine word costs two steps here and four there.

The state is stored one step ahead of the position, so that a draw returns a pair the previous draw already computed and the processor overlaps the output with the next state. That is internal: seeds, `get_state`, `set_state!` and show all speak the same representation as `MRG32k3a`.

Examples

```
julia> rng = MRG63k3a();
```

```
julia> rand(rng)
0.999964376179128
```

[source](#)

`RandomDataStreams.MRG63k3aGen` – Type.

An object that generates independent `MRG63k3a` streams, 2^{250} numbers apart.

[source](#)

`RandomDataStreams.checkseed63` – Function.

```
checkseed63(x::Vector{Int}) -> Bool
```

True when `x` is a valid `MRG63k3a` seed: six non-negative integers, the first three below `m1 = 2^63 - 6645` and not all zero, the last three below `m2 = 2^63 - 21129` and not all zero.

[source](#)

5.4 Xoshiro / xoroshiro families

`RandomDataStreams.LinRNG` – Type.

```
LinRNG{N,S} <: AbstractStreamableRNG
```

Generic xoshiro/xoroshiro generator: state of `N` 64-bit words and scrambler `S ∈ (:plus, :starstar, :plusplus)`. Holds three checkpoints — current state (`Cg`), substream start (`Bg`) and stream start (`Ig`) — enabling `reset_stream!`, `reset_substream!` and anchored jumps. Use the exported aliases (`Xoroshiro128p`, ..., `Xoshiro512pp`) instead.

[source](#)

`RandomDataStreams.Xoroshiro128p` – Type.

Xoroshiro128p <: **AbstractStreamableRNG**

xoroshiro128+ (Blackman & Vigna): 128-bit state, period $2^{128} - 1$, output $s_0 + s_1$. Fastest small-state generator for floating-point use; the four lower bits have low linear complexity and it has a very mild Hamming-weight dependency after ~5 TB of output. Suitable only for mild parallelism (2^{32} streams x 2^{64} substreams).

[source](#)

`RandomDataStreams.Xoroshiro128ss` – Type.

Xoroshiro128ss <: **AbstractStreamableRNG**

xoroshiro128** (Blackman & Vigna): 128-bit state, period $2^{128} - 1$, all-purpose scrambler ($\text{rotr}(s_1 * 5, 7) * 9$), 1-dimensionally equidistributed. Suitable only for mild parallelism (2^{32} streams x 2^{64} substreams).

[source](#)

`RandomDataStreams.Xoroshiro128pp` – Type.

Xoroshiro128pp <: **AbstractStreamableRNG**

xoroshiro128++ (Blackman & Vigna): 128-bit state, period $2^{128} - 1$, all-purpose scrambler ($\text{rotr}(s_0 + s_1, 17) + s_0$). Same parallelism limits as the other 128-bit variants.

[source](#)

`RandomDataStreams.Xoshiro256p` – Type.

Xoshiro256p <: **AbstractStreamableRNG**

xoshiro256+ (Blackman & Vigna): 256-bit state, period $2^{256} - 1$, output $s_0 + s_3$. Slightly faster than `**/++` when generating only floating-point numbers from the upper bits; avoid for raw 64-bit consumption.

[source](#)

`RandomDataStreams.Xoshiro256ss` – Type.

Xoshiro256ss <: **AbstractStreamableRNG**

xoshiro256** (Blackman & Vigna): 256-bit state, period $2^{256} - 1$, all-purpose, 3-dimensionally equidistributed (default PRNG of Lua and .NET 6).

[source](#)

`RandomDataStreams.Xoshiro256pp` – Type.

Xoshiro256pp <: **AbstractStreamableRNG**

xoshiro256++ (Blackman & Vigna): 256-bit state, period $2^{256} - 1$, all-purpose, 3-dimensionally equidistributed (default PRNG of Julia and Rust's `SmallRng`). The recommended general-purpose variant.

Examples

```
julia> rng = Xoshiro256pp(fill(UInt64(42), 4));
```

```
julia> rand(rng, UInt64)
```

```
0x000000002a00002a
```

[source](#)

`RandomDataStreams.Xoshiro512p` – Type.

Xoshiro512p <: **AbstractStreamableRNG**

xoshiro512+ (Blackman & Vigna): 512-bit state, period $2^{512} - 1$, output $s_0 + s_2$. Floating-point oriented; same caveats as Xoshiro256p.

[source](#)

`RandomDataStreams.Xoshiro512ss` – Type.

Xoshiro512ss <: **AbstractStreamableRNG**

xoshiro512** (Blackman & Vigna): 512-bit state, period $2^{512} - 1$, all-purpose, 7-dimensionally equidistributed.

[source](#)

`RandomDataStreams.Xoshiro512pp` – Type.

Xoshiro512pp <: **AbstractStreamableRNG**

xoshiro512++ (Blackman & Vigna): 512-bit state, period $2^{512} - 1$, all-purpose. Offers 2^{256} substreams of 2^{256} values — more than any application needs; prefer Xoshiro256pp unless you have a specific reason.

[source](#)

Stream generators

`RandomDataStreams.LinGen` – Type.

LinGen{N,S} <: **AbstractRNGStream**

Stream generator producing non-overlapping streams of `LinRNG`{N,S}: each call to `next_stream!` applies the family's long jump to the stored seed. Use the exported aliases (`Xoroshiro128pGen`, ..., `Xoshiro512ppGen`) instead.

[source](#)

`RandomDataStreams.Xoroshiro128pGen` – Type.

Xoroshiro128pGen <: **AbstractRNGStream**

Stream generator minting non-overlapping Xoroshiro128p streams via a long jump (2^{96} values) per call. Seeds are 2 `UInt64` words.

[source](#)

`RandomDataStreams.Xoroshiro128ssGen` – Type.

Xoroshiro128ssGen <: **AbstractRNGStream**

Stream generator minting non-overlapping Xoroshiro128ss streams via a long jump (2^{96} values) per call. Seeds are 2 `UInt64` words.

[source](#)

RandomDataStreams.Xoroshiro128ppGen - Type.

```
Xoroshiro128ppGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoroshiro128pp streams via a long jump (2^{96} values) per call. Seeds are 2 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro256plusGen - Type.

```
Xoshiro256plusGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256p streams via a long jump (2^{192} values) per call. Seeds are 4 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro256pGen - Type.

```
Xoshiro256pGen <: AbstractRNGStream
```

Regular spelling of [Xoshiro256plusGen](#), matching Xoshiro256ssGen, Xoshiro512pGen and the rest. The two names are the same type.

[source](#)

RandomDataStreams.Xoshiro256ssGen - Type.

```
Xoshiro256ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256ss streams via a long jump (2^{192} values) per call. Seeds are 4 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro256ppGen - Type.

```
Xoshiro256ppGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256pp streams via a long jump (2^{192} values) per call. Seeds are 4 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro512pGen - Type.

```
Xoshiro512pGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro512p streams via a long jump (2^{384} values) per call. Seeds are 8 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro512ssGen - Type.

```
Xoshiro512ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro512ss streams via a long jump (2^{384} values) per call. Seeds are 8 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro512ppGen – Type.

Xoshiro512ppGen <: **AbstractRNGStream**

Stream generator minting non-overlapping Xoshiro512pp streams via a long jump (2^{384} values) per call. Seeds are 8 UInt64 words.

[source](#)

5.5 Counter-based generators

RandomDataStreams.CBGen – Type.

CBGen{**B**,**W**,**N**,**K**}

Hands out independent CBRNG streams, each with its own key. Concrete aliases (PhiloxGen, Philox4x64Gen, ...) are what user code normally names.

The generator walks an odometer over the seeds 0, 1, 2, ... and maps each one through the family's key schedule `stream_key` to obtain the actual key; see that function for why the two are not the same thing.

[source](#)

Functions

Base.rand – Method.

Produces a raw random number with 32 bits of precision.

[source](#)

Base.rand – Method.

Produces a raw random number with 63 bits of precision, of which a Float64 keeps 53.

[source](#)

Base.rand – Method.

Return a random Float64 in [0, 1).

[source](#)

Base.rand – Method.

Generates a Float32 from any xoshiro/xoroshiro generator.

[source](#)

Base.rand – Method.

Generates a Float16 from any xoshiro/xoroshiro generator.

[source](#)

`Base.rand` – Method.

Return a random Float64 in [0, 1).

[source](#)

`Base.rand` – Method.

Generates a Float32 from any counter-based generator.

[source](#)

`Base.rand` – Method.

Generates a Float16 from any counter-based generator.

[source](#)

`Base.rand` – Method.

Return a random Float64 in [0, 1).

[source](#)

`Base.rand` – Method.

Generates a Float32 from a PCG generator.

[source](#)

`Base.rand` – Method.

Generates a Float16 from a PCG generator.

[source](#)

`Base.rand` – Method.

Hook required by Random's generic machinery for Float64-native generators: produces a Float64 in [1, 2). Everything else (ints, arrays, shuffle...) derives from this without further plumbing.

[source](#)

`Base.rand` – Method.

Hook required by Random's generic machinery for Float64-native generators: produces a Float64 in [1, 2). Everything else (ints, arrays, shuffle...) derives from this without further plumbing.

[source](#)

`RandomDataStreams.short_jump!` – Function.

Jumps forward by 2^{96} values (xoroshiro128), 2^{128} (xoshiro256) or 2^{256} (xoshiro512): the start of the next non-overlapping substream.

[source](#)

Jumps forward by $2^{64} + 1$ values: the start of the next non-overlapping substream. The distance is odd on purpose; see the implementation notes.

[source](#)

`RandomDataStreams.long_jump!` – Function.

Jumps forward by 2^{96} (xoroshiro128), 2^{192} (xoshiro256) or 2^{384} (xoshiro512) values: the start of the next independent stream.

[source](#)

Jumps forward by $2^{32} * (2^{64} + 1) + 1$ values: the start of the next independent stream. The distance is odd on purpose; see the implementation notes.

[source](#)

`RandomDataStreams.advance_state!` – Function.

Given a random number generator, jumps n steps forward if $n > 0$ (or $-n$ steps backwards if $n < 0$), where if $e > 0$, let $n = 2^e + c$; if $e < 0$, let $n = -2^{(-e)} + c$; if $e = 0$, let $n = c$.

Examples

```
julia> rng = MRG32k3a();

julia> advance_state!(rng, Int64(2), Int64(-1)); # skip n = 2^2 - 1 = 3 values

julia> rand(rng)
0.8258468629271136
```

[source](#)

Given a random number generator, jumps n steps forward if $n > 0$ (or $-n$ steps backwards if $n < 0$), where if $e > 0$, let $n = 2^e + c$; if $e < 0$, let $n = -2^{(-e)} + c$; if $e = 0$, let $n = c$.

Examples

```
julia> rng = MRG63k3a();

julia> advance_state!(rng, Int64(2), Int64(-1)); # skip n = 2^2 - 1 = 3 values

julia> rand(rng)
0.8707612110911583
```

[source](#)

Given a random number generator, jumps n steps forward if $n > 0$ (or $-n$ steps backwards if $n < 0$), where if $e > 0$, let $n = 2^e + c$; if $e < 0$, let $n = -2^{(-e)} + c$; if $e = 0$, let $n = c$.

The distance is reduced modulo the period $(2^{(64N)} - 1)$, so any magnitude is accepted. Only the current position moves; the stream/substream boundaries (Bg, Ig) are left untouched. Costs $O((64N)^2)$ GF(2) operations.

[source](#)

Given a random number generator, jumps n steps forward if $n > 0$ (or $-n$ steps backwards if $n < 0$), where if $e > 0$, let $n = 2^e + c$; if $e < 0$, let $n = -2^{(-e)} + c$; if $e = 0$, let $n = c$.

The distance is reduced modulo the period 2^{128} , so any magnitude is accepted and a backward jump costs the same as a forward one. Only the current position moves; the stream and substream boundaries are left untouched.

[source](#)

Given a random number generator, jumps n words forward if $n > 0$ (or $-n$ words backwards if $n < 0$), where if $e > 0$, let $n = 2^e + c$; if $e < 0$, let $n = -2^{(-e)} + c$; if $e = 0$, let $n = c$.

The unit is one `rand(rng)` draw, the same as for every other generator of the package. A draw takes 64 bits, so it consumes two words of a 32-bit family and one word of a 64-bit one. The distance is reduced modulo the counter space, so any magnitude is accepted. The stream and substream boundaries are not affected, only the current position.

[source](#)

`RandomDataStreams.srand!` – Function.

```
srand!(gen::MRG32k3aGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

[source](#)

```
srand!(rng::MRG32k3a, seed) -> rng
```

Reseeds the generator, resetting the stream and substream boundaries to seed.

[source](#)

```
srand!(gen::MRG63k3aGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

[source](#)

```
srand!(rng::MRG63k3a, seed) -> rng
```

Reseeds the generator, resetting the stream and substream boundaries to seed.

[source](#)

Seeds a generator with the first words of seed.

[source](#)

```
srand!(gen, seed::Vector{UInt64}) -> gen
```

Resets the seed that the next `next_stream!` call will use.

[source](#)

Seeds a generator, resetting the stream and substream boundaries to the seed.

[source](#)

```
srand!(gen::PCGGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

[source](#)

```
srand!(rng::CBRNG, key) -> rng
```

Reseeds the stream with key (a tuple or a vector of key words) and rewinds its counter to the start of the stream.

source

```
srand!(gen::CBGen, seed) -> gen
```

Resets the seed that the next `next_stream!` call will use.

source

`RandomDataStreams.get_state` – Function.

`get_state` return the seed used to generate non-overlapping MRGs

source

```
get_state(rng::MRG32k3a) -> Vector{Int}
```

Copy of the current state (Cg); restore it with `set_state!`.

source

```
get_state(gen::MRG63k3aGen) -> Vector{Int}
```

The seed the next `next_stream!` will use; restore it with `set_state!`.

source

```
get_state(rng::MRG63k3a) -> Vector{Int}
```

The current position, in the same representation as a seed; restore it with `set_state!`. The internal vector runs one step ahead of it, so this steps back by one – a matrix-vector product, off any hot path.

source

```
get_state(rng::LinRNG) -> Vector{UInt64}
```

Copy of the current state (Cg); restore it into a fresh generator via its three-argument constructor.

source

```
get_state(gen) -> Vector{UInt64}
```

Seed that will be used by the next `next_stream!` call.

source

```
get_state(rng::PCGRNG) -> UInt128
```

Current state. Hand it back to `set_state!` to return here.

source

```
get_state(gen::PCGGen) -> UInt128
```

Seed that will be used by the next `next_stream!` call.

source

```
get_state(rng::CBRNG) -> (ctr, key, buffer, idx)
```

Full internal state: the index of the next block to produce, the key, the block most recently produced and the position of the next word to consume in it.

[source](#)

```
get_state(gen::CBGen) -> NTuple{K,W}
```

Seed that the next `next_stream!` call will use. This is the position in the key schedule, not the key itself; the two coincide only when `stream_key` is the identity.

[source](#)

`RandomDataStreams.set_state!` – Function.

```
set_state!(gen::MRG32k3aGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

[source](#)

```
set_state!(rng::MRG32k3a, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current state moves; the stream and substream boundaries (`Bg`, `Ig`) are untouched.

[source](#)

```
set_state!(gen::MRG63k3aGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

[source](#)

```
set_state!(rng::MRG63k3a, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current state moves; the stream and substream boundaries (`Bg`, `Ig`) are untouched.

[source](#)

```
set_state!(rng::LinRNG, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current position moves; the stream and substream boundaries (`Bg`, `Ig`) are untouched.

[source](#)

```
set_state!(gen::LinGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

[source](#)

```
set_state!(rng::PCGRNG, state) -> rng
```

Restores the current position from a `get_state(rng)` value. Only the current position moves; the stream and substream boundaries are untouched.

source

```
set_state!(gen::PCGGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

source

```
set_state!(rng::CBRNG, state) -> rng
```

Restores the current position of the stream from a `get_state(rng)` value: the counter, the key, the buffered block and the index within it.

source

```
set_state!(gen::CBGen, seed) -> gen
```

Restores the seed of the next stream, as returned by `get_state(gen)`.

source

`RandomDataStreams.next_stream!` – Function.

```
next_stream!(gen, n::Integer) -> Vector
```

Return `n` successive non-overlapping streams from `gen`, in order.

This is the thread-safe way to obtain streams for parallel work. A generator object holds the seed of the next stream and rewrites it on every call, so calling `next_stream!(gen)` from several threads at once is a data race: the threads read the same seed and hand out streams that overlap. The failure is silent — the result is not an error but a set of correlated streams, which is exactly the guarantee the package exists to provide.

Call this once, from one thread, and give each worker its own element:

```
using Base.Threads

gen = MRG32k3aGen()
rngs = next_stream!(gen, nthreads()) # serial, here

@threads for t in 1:nthreads()
    r = rngs[t] # each thread owns one stream
    ...
end
```

Streams themselves carry no shared state, so drawing from different streams on different threads needs no synchronisation.

source

Given an RNG generator object, returns the next RNG stream.

source

Given an RNG generator object, returns the next RNG stream.

source

Given an RNG generator object, returns the next RNG stream.

source

Given an RNG generator object, returns the next RNG stream.

source

```
next_stream!(gen) -> rng
```

Returns the next independent stream and moves the generator on to the following seed. Seeds are walked in lexicographic order and mapped through `stream_key`, a bijection, so distinct streams never share a key and their counter spaces cannot overlap.

source

`RandomDataStreams.reset_stream!` – Function.

Resets a given random number generator to the beginning of the current stream.

source

Resets a given random number generator to the beginning of the current stream.

source

```
reset_stream!(rng) -> rng
```

Rewind the generator to the very beginning of its current stream ($C_g = B_g = I_g$).

source

```
reset_stream!(rng) -> rng
```

Rewind the generator to the very beginning of its current stream.

source

```
reset_stream!(rng) -> rng
```

Rewind the generator to the very beginning of its current stream (counter 0).

source

`RandomDataStreams.reset_substream!` – Function.

Resets a given random number generator to the beginning of the current substream.

source

Resets a given random number generator to the beginning of the current substream.

source

```
reset_substream!(rng) -> rng
```

Rewind the generator to the beginning of its current substream ($C_g = B_g$).

source

```
reset_substream!(rng) -> rng
```

Rewind the generator to the beginning of its current substream.

[source](#)

```
reset_substream!(rng) -> rng
```

Rewind the generator to the beginning of its current substream.

[source](#)

`RandomDataStreams.next_substream!` – Function.

Takes a random number generator and shifts seed to next substream.

[source](#)

Takes a random number generator and shifts seed to next substream, 2^{150} numbers ahead.

[source](#)

```
next_substream!(rng) -> rng
```

Move the generator to the start of the next substream (anchored jump from Bg): 2^{64} values for xoroshiro128, 2^{128} for xoshiro256, 2^{256} for xoshiro512.

[source](#)

```
next_substream!(rng) -> rng
```

Move the generator to the start of the next substream: an anchored jump of $2^{64} + 1$ values from the current substream boundary.

[source](#)

```
next_substream!(rng) -> rng
```

Move the generator to the start of the next substream: a jump of $2^{(\text{_substream_shift})}$ blocks in the counter (2^{64} blocks for Philox4x32-10).

[source](#)

Part VI

Implementation Notes

Chapter 6

Implementation Notes

6.1 MRG32k3a

MRG32k3a is L'Ecuyer's combined multiple recursive generator of order 3. It maintains two 3-component integer states updated by the recurrences

```
p1 = (1403580 · x2 - 810728 · x1) mod m1,    m1 = 232 - 209
p2 = (527612 · y2 - 1370589 · y0) mod m2,    m2 = 232 - 22853
u  = (p1 - p2) / m1    (with wrap-around) ∈ [0, 1)
```

The combined generator has period $\approx 2^{191}$.

Modular arithmetic without overflow, and without tests

The recurrence follows Vigna's testless formulation (<https://github.com/vigna/MRG32k3a>). The two negative coefficients are carried as positive numbers and subtracted, and a multiple of the modulus is added, so the argument of % can never be negative and the residual needs no correction afterwards; the combination $z = p1 - p2 \bmod m1$ is done with an arithmetic shift rather than a comparison. Worth about 15% here — 204 million Float64 per second before, 230 to 235 after depending on the run — and the stream is unchanged bit for bit.

It is worth 15% and not the factor of two Vigna reports, because most of what he removes by hand LLVM already removes for us: the previous formulation, with its two residual corrections and its comparison, compiled to 72 instructions with **no branch and no division** — the corrections had become conditional moves and the constant moduli multiply-shift sequences. What is left to gain is the fix-up a *signed* remainder needs, which disappears once the dividend is known non-negative: 72 instructions become 68.

Two variants measured and rejected: keeping the state in six scalar fields instead of a `Vector{Int64}` is *slower* (226 against 235) while breaking the constructors and `Cg`; and computing the output from the current state before the update, which needs the state kept one step ahead to preserve the stream, gives 231 and changes what `get_state` and the jump matrices operate on.

That second variant is Vigna's third optimization, and the conclusion does **not** carry over to MRG63k3a, where it is worth 7% and is used — see below. Re-measured here with the same harness, MRG32k3a gives 236 against 235: the 32-bit step is short enough that the combination is already hidden behind it, so there is nothing to overlap.

Modular arithmetic in the jump machinery

All state components stay below $m1 < 2^{32}$, so products fit in Float64 exactly ($< 2^{53}$). The helper `MultModM(a, s, c, m)` computes $(a \cdot s + c) \bmod m$ using this fact and falls back to a split-by- 2^{17} computation when intermediate values would exceed 2^{53} . Matrix helpers:

- `MatVecModM(A, s, m)` — matrix-vector product mod m
- `MatMatModM(A, B, m)` — matrix product mod m
- `MatTwoPowModM(A, e, m)` — A raised to the power 2^e
- `MatPowModM(A, n, m)` — A^n by binary exponentiation

Streams, substreams, jumps

Moving a stream forward by k steps is a linear operation on its state, hence a precomputed matrix power applied to the state vector:

Operation	Matrix	Step
<code>next_stream!(gen)</code>	$A1p127, A2p127$	2^{127} values
<code>next_substream!(rng)</code>	$A1p76, A2p76$	2^{76} values
<code>advance_state!(rng, e, c)</code>	$\text{MatTwoPowModM} / \text{Inv}A1, \text{Inv}A2$	arbitrary (inverse matrices allow backward jumps)

The RNG keeps three checkpoints: C_g (current), B_g (substream start), I_g (stream start), which is what makes `reset_substream!` and `reset_stream!` $O(1)$.

6.2 MRG63k3a

The same construction as MRG32k3a, sized for 64-bit arithmetic: the fourth entry of Table II in L'Ecuyer (1999), which he implements in C as MRG63k3a.

```
p1 = (1754669720 · x2 - 3182104042 · x0) mod m1,    m1 = 2^63 - 6645
p2 = (31387477935 · y2 - 6199136374 · y0) mod m2,    m2 = 2^63 - 21129
u = (p1 - p2) / (m1 + 1)    (with wrap-around) ∈ (0, 1)
```

Period $\approx 2^{377}$, against 2^{191} ; entropy per step $\log_2(m1) = 63.0$ bits, against 32.0. The package reproduces L'Ecuyer's C implementation value for value, checked on three seeds and a million draws.

Reduction without a 128-bit division

The products no longer fit in a `Float64` mantissa — that is the whole reason this generator is a separate implementation and not a parameter set — so they are computed with `widemul` into an `Int128`. What must be avoided then is the remainder: a 128-bit `%` is a software routine, and it would dominate the step.

Both moduli are pseudo-Mersenne, $m = 2^{63} - c$ with c small, so the reduction is two shifts, a multiply and an add. Splitting the 128-bit value at bit 63 as $t = hi \cdot 2^{63} + lo$ gives

$$t \equiv hi \cdot c + lo \pmod{m}, \quad \text{since } 2^{63} \equiv c \pmod{m}.$$

With the same testless formulation as MRG32k3a — the negative coefficient carried positive and a multiple of the modulus added — t stays below 2^{99} , so $hi < 2^{36}$ and $hi \cdot c < 2^{51}$: the folded value is below $2^{63} + 2^{51}$, which is under $2m$, and one conditional subtraction finishes it. The combination $z = p1 - p2 \bmod m1$ then uses the same arithmetic shift as MRG32k3a. The whole step compiles to 61 instructions with no division and no branch — the two conditional subtractions become conditional moves.

The reference C code takes a different route to the same values, Bratley, Fox & Schrage's approximate factoring ($q = m \div a$, $r = m \bmod a$), which keeps every intermediate inside a 64-bit signed integer at the cost of two

divisions and two branches per coefficient. That is the right choice for a 1999 C compiler and the wrong one here: the reduction above is what makes MRG63k3a *faster* than MRG32k3a per random bit rather than slower. The alternative was measured, not assumed — a plain 128-bit remainder instead of the fold runs at 19 million draws per second against 196, a factor of ten.

The state runs one step ahead

Vigna's three optimizations for MRG32k3a are the testless formulation above, the branchless combination, and computing the output from the *current* state so that the processor overlaps it with the next state. The first two are shared with MRG32k3a; the third is used **here only**, because it is the one place where the two generators measure differently:

	MRG32k3a	MRG63k3a
output from the new state	236 M/s	183 M/s
output from the current state	235 M/s	196 M/s

The 63-bit step is a `widemul` and a fold, long enough that the combination sitting at the end of it is visible; the 32-bit step is a `Float64` multiply and a compiler-optimised `%`, short enough that it is not. So `next_pair!` returns the pair already in `Cg[3]`, `Cg[6]` and computes the next one, and the state vector is one step ahead of the position.

That shift is invisible from outside. It costs nothing anywhere it might have: the jump matrices commute with the one-step matrix, so `advance_state!`, `next_substream!`, `next_stream!` and the `Bg/Ig` checkpoints operate on the shifted vector unchanged, and jumping a shifted state gives the shifted jumped state. Only the boundary converts — the constructors and `Random.seed!` step a seed forward, `get_state` and `show` step the stored vector back — at one matrix-vector product each, on paths that are never hot. The stream is identical bit for bit, which is what the reference vectors from L'Ecuyer's C code check.

Jump matrices

`MultModM` here is the straightforward `widemul` plus 128-bit mod, since it serves only the jump machinery, never the draw. The matrix helpers are the same four as for MRG32k3a.

The jump distances are **not** L'Ecuyer's: he published `A1p76/A1p127` for MRG32k3a and nothing for MRG63k3a. This package uses 2^{150} between substreams and 2^{250} between streams — his 2^{76} and 2^{127} scaled by the ratio of the two periods, $377/191 \approx 1.97$ — which cuts the period into 2^{127} streams of 2^{100} substreams of 2^{150} numbers each. The four matrices are computed by repeated squaring at precompilation rather than tabulated, roughly 400 matrix products, and the test suite checks the arithmetic that produces them: $A \cdot \text{InvA} = I$ for both components, $(A^{(2^{150})})^{(2^{100})} = A^{(2^{250})}$, `next_substream!` equal to `advance_state!(rng, 150, 0)`, and `next_stream!` equal to `advance_state!(rng, 250, 0)`. The formulas behind `InvA1` and `InvA2` are checked by a route that does not depend on this generator at all: applied to MRG32k3a's coefficients they must reproduce the inverse matrices L'Ecuyer tabulates, and they do.

Integer outputs

Same construction as MRG32k3a, one size up: $z \in [1, m1]$ is truncated to 32-bit chunks and wider words are concatenations of consecutive chunks. A `UInt64` is two steps, against four; a `UInt32` is one, against two. Because $m1 = 2^{63} - 6645$, a residue class modulo 2^{32} holds 2^{31} values give or take one, so the departure from uniformity is of order 2^{-31} per chunk against 2^{-16} for MRG32k3a. Still not exact — no exact uniform word comes out of a non-power-of-two modulus without rejection — but two orders of magnitude closer, at half the cost.

Float64 remains the native output (`Random.rng_native_52`): one step covers a double, where declaring the word native would make every double cost two.

The one-step-ahead ordering helps the paths whose critical path ends in the combination — `Float64` and `UInt32` — and not `UInt64`, where two steps run back to back and the recurrence, not the combination, is the chain.

6.3 Xoshiro256+

`xoshiro256+` (Blackman & Vigna, 2019) keeps four 64-bit words and produces one word per step with only shifts, XORs and rotations:

```
result = s1 + s4
t = s2 << 17
s3 = xor(s3, s1); s4 = xor(s4, s2); s2 = xor(s2, s3); s1 = xor(s1, s4); s3 = xor(s3, t); s4 =
↪ rotl(s4, 45)
```

The output word is the sum of two internal words (+ variant): fastest of the xoshiro family for float generation, though the low three bits of the raw output have limited linear complexity — irrelevant once scaled to `Float64` (53 bits used). Period: $2^{256} - 1$.

Jumps

`short_jump!` and `long_jump!` use Vigna's published jump polynomials. They are computed as XOR-linear combinations of states visited while stepping through the 256 bits of each jump constant. `RandomDataStreams.jl` hoists the jump constants into compile-time tuples and evaluates jumps allocation-free on immutable state tuples.

`RandomDataStreams` semantics differ subtly from the reference C code: jumps are **anchored** at stream boundaries. `short_jump!(rng)` first resets $C_g \leftarrow B_g$, then applies the jump polynomial, and stores the result in both C_g and B_g ; `long_jump!(rng)` does the same with respect to I_g . This matches L'Ecuyer's stream model (`next_substream!/reset_stream!`) and makes scenario replay reproducible regardless of consumption inside the current substream.

Families and scramblers

The package implements all 64-bit variants from `xoshiro.di.unimi.it` through a single generic type `LinRNG{N, S}` (state words $\times$ scrambler):

- `transition_lin_step` for `NTuple{2}` (`xoroshiro128`: $a=24, b=16, c=37$), `NTuple{4}` (`xoshiro256`) and `NTuple{8}` (`xoshiro512`);
- scramblers `+`, `**` (`rotr(s2·5, 7)·9`) and `++` (`rotr(s1+s4, 23)+s1` for 256; family-specific constants);
- per-family jump constants shared by all scramblers.

Exported aliases: `Xoroshiro128p/ss/pp`, `Xoshiro256p/ss/pp`, `Xoshiro512p/ss/pp` plus matching `*Gen` stream generators.

All nine variants are regression-tested against byte-exact outputs produced by the original C implementations compiled with `gcc` (sequences, short jumps and long jumps).

Arbitrary forward/backward jumps

Because the transition is multiplication by x in $\text{GF}(2)[x]/p(x)$ — with p the family's characteristic polynomial, as published in Vigna's reference files — one can jump any distance n in constant time by applying the polynomial $x^n \bmod p$ through the same accumulate-and-step loop as the fixed jumps. Backward distances use the multiplicative order of x : $x^{(-n)} = x^{(2^{\text{deg}}-1-n)}$ ($\text{deg} = 64N$). `RandomDataStreams.jl` implements a small $\text{GF}(2)$ polynomial engine (`_poly_mul_mod`, `_poly_pow_x`, `BigInt` exponents reduced modulo the period) and exposes it as `advance_state!(rng, e, c)` with the exact distance convention of MRG32k3a. Correctness is property-tested: fixed jumps coincide with `short_jump!`, round trips (+ k then $-k$) restore the state bit-for-bit, and backward-jumped generators re-emit previously seen values.

State representation

The state lives in `NTuple{4, UInt64}` fields. Immutable tuples let the JIT keep the whole working set in registers inside `next`, eliminating bounds checks and heap traffic that a `Vector{UInt64}` representation incurs. Measured throughput: $\approx 10^9$ draws/s on a single core.

6.4 PCG

`PCGRNG{V}` runs a linear congruential recurrence on 128 bits, $s \leftarrow a*s + c \pmod{2^{128}}$, and permutes the state into 64 bits of output. Two variants ship: `PCG64` with the XSL-RR permutation, which is what `numpy.random.default_rng()` returns, and `PCG64DXSM` with the DXSM permutation and the cheap 64-bit multiplier, which NumPy recommends for large-scale parallel work. They differ in one more detail that matters for bit-exactness: `PCG64` steps the state and permutes the result, `DXSM` permutes the state and then steps it. Both are pinned by known-answer vectors taken from NumPy.

Jumps are closed-form. For an LCG,

$$s_n = a^n * s_0 + c * (a^n - 1) / (a - 1) \pmod{2^{128}},$$

which Brown's (1994) doubling recurrence evaluates in $O(\log n)$ multiplications. `advance_state!` therefore costs the same at any distance, and a backward jump of n is the forward jump of $2^{128} - n$, so it costs the same as a forward one. This is the cheapest arbitrary jump in the package: the xoshiro families pay $O(\text{deg}^2)$ $\text{GF}(2)$ operations for the same operation, tens to hundreds of milliseconds, and MRG32k3a a sequence of matrix products. **The jump distances are odd, and that is not cosmetic.** The obvious choice — 2^{64} between substreams, 2^{96} between streams, mirroring the xoshiro128 layout — is wrong for a linear congruential recurrence. By the lifting-the-exponent lemma,

$$v_2(a^n - 1) = v_2(a - 1) + v_2(a + 1) + v_2(n) - 1 = 2 + v_2(n) \quad (n \text{ even}),$$

so a jump of 2^m produces a jump multiplier congruent to 1 modulo $2^{(m+2)}$. With $m = 96$, the states of successive streams agree in their low 98 bits up to an arithmetic progression. Each stream still passes `SmallCrush` on its own; interleaving 64 of them round-robin fails it outright, with 14 of 15 p-values at 0. The package's inter-stream battery found this, which is the argument for having one.

An odd distance has $v_2(n) = 0$, leaving the jump multiplier as far from the identity as the multiplier itself. Substreams are therefore $2^{64} + 1$ apart and streams $2^{32} * (2^{64} + 1) + 1$, which is still 2^{32} streams of 2^{32} substreams of 2^{64} draws and still non-overlapping — the last substream of a stream ends two values below the next stream's start. With those distances the interleaved battery passes. Both the parity of the distances and the 2-adic distance of the resulting jump multipliers are asserted in the test suite.

The increment is fixed, and that is deliberate. PCG has its own notion of a stream: every odd increment c gives a full-period orbit, and NumPy exposes it as part of the state. Those orbits are not known to be independent. Writing $t_n = s_n + h$ and matching the two recurrences gives

$$h * (a - 1) = c1 - c2,$$

which has a solution whenever 4 divides $c1 - c2$ — the full-period condition forces $a \equiv 1 \pmod{4}$, and $v2(a - 1) = 2$ for the PCG multipliers. For half of all pairs of odd increments, then, the two "independent streams" are the same state sequence translated by a constant. Taking $c2 = c1 - (a - 1)$ makes that constant 1, and the two output sequences then differ by a mean Hamming distance of 2 bits out of 64, against 32 for unrelated sequences, with 94% of draws differing in at most 4 bits.

That is an adversarial pair; most pairs of increments are far less structured and would show nothing. The objection is not that every pair is bad but that no criterion is published for telling the good pairs from the bad ones, which is exactly what L'Ecuyer et al. (2021, Sec. 3) argue against — the same objection they raise against Squares. So the package fixes the increment to the reference value for every PCG stream and derives streams from jumps, whose non-overlap is a statement about distances along one orbit. The algebra above is locked in the test suite.

6.5 Counter-based generators

CBRNG{B,W,N,K} holds what every counter-based family shares — the counter, the key, the block most recently produced and the index of the next word in it. This is the state (k, i, j) of L'Ecuyer et al. (2021, Sec. 3), with $f(k, i, j) = (k, i + I[j = d-1], (j + 1) \bmod d)$ and $d = N$. A variant only supplies `bijection(::Val{B}, ctr, key)`.

Two families are built on it. **Philox** keeps both halves of a fixed-constant multiplication and xors the high halves into the other words, with a Weyl sequence for the round key. **Threefry** is a reduction of the Threefish cipher used in Skein and contains no multiplication at all — add, rotate, xor only — so it is reproducible bit-for-bit on any architecture, including ones with no fast integer multiply. Adding it required only its bijection and its aliases, no stream machinery.

Its rounds are unrolled at compile time with a generated function. Each round uses a different pair of Skein rotation constants, so a plain loop indexes the constant table with a value LLVM cannot fold; unrolling is worth a factor of 5.8 on the bijection ($7.7 \rightarrow 44.8$ Mblocks/s), which is the difference between Threefry being unusable and being in the same class as Philox. The round count is a parameter of the bijection, not of the type, so the faster 13-round variant is three lines of user code.

Streams are keys. **Substreams** partition the counter, following the same paper: "one can use the $c0 < c$ most significant bits of the counter to determine the substream, and the remaining $c1 = c - c0$ bits for the position within the substream". `_substream_shift` splits the counter in half, so Philox4x32-10 has 2^{64} substreams of 2^{64} blocks each.

Key schedules. CBRNG walks the seeds $0, 1, 2, \dots$ and maps each through `stream_key`, a bijection that defaults to the identity. Consecutive small keys are safe for Philox — the ten rounds of encoding absorb the structure, as reported by Salmon et al. (2011) and discussed in L'Ecuyer et al. (2021, Sec. 3) — but they are *not* safe for every counter-based construction, so a family whose bijection is weak for structured keys must override `stream_key` with a schedule that hashes the seed.

6.6 Integer outputs of MRG32k3a

MRG32k3a returns $z = (x1 - x2) \bmod m1$ with $m1 = 2^{32} - 209$, prime, scaled to $u = z / (m1 + 1)$. Its native output is a uniform integer over a range that is **not** a power of two, carrying $\log_2(m1) = 32.0$ bits. This is why the reference literature — L'Ecuyer (1999), and the RNGStreams package of L'Ecuyer, Simard, Chen & Kelton (2002) whose stream API this package follows — exposes only `RandU01()` and `RandInt(i, j) = i + floor((j - i + 1) * RandU01())`, and no native-word or raw-bit primitive: no exact uniform word comes out of one draw without rejection.

Machine integers are therefore an extension beyond what that literature specifies and validates, and they are built here by assembling 16-bit chunks of z , one MRG step per chunk — four steps for a `UInt64`. Truncating z to 16 bits carries a relative bias of 2^{-16} , since 2^{16} does not divide $m1$; taking the low bits is sound because the modulus is prime, so there is no low-bit weakness of the kind a power-of-two LCG has.

The cheaper alternative — assembling a word from the mantissas of two $[1, 2)$ draws, two steps instead of four — was implemented here and removed. It assumes 52 random bits per double, and MRG32k3a supplies 32; the low mantissa bits are a near-deterministic function of the high ones. The result passed every $U(0, 1)$ battery and failed SmallCrush on its own bit stream with six p -values at zero, bit 12 coming out set in two draws out of three. `test/test_bits.jl` is the regression net.

Narrower types are truncations of one chunk. None of these integers is exactly uniform over its full width — fine for indexing, shuffling and flags in a simulation, not for cryptographic use. MRG63k3a above is the same construction with twice the chunk width and half the steps per word; it is the one to reach for when a run consumes integers rather than floats.

6.7 Testing strategy

Reference sequences were captured from the reference implementations (L'Ecuyer's C code for MRG32k3a and MRG63k3a — for the latter, ten doubles, the draw one million steps later, and the raw combined integer on three seeds; Vigna's constants for the xoshiro jumps) and regression-checked after optimization: identical outputs are produced for every supported type, plus `reset/jump/state` round-trips.

6.8 Performance snapshot

Reproduce with `julia -O3 scripts/benchmarks/throughput.jl`, which prints the Julia version and CPU it ran on. On a hybrid CPU (Intel 12th generation and later), pin the process first — `taskset -c 0 julia -O3 ...` — and say which core type was used: otherwise the scheduler moves the run between performance and efficiency cores and the minimum reflects whichever was fastest. Figures below are one run on a Skylake `x86_64`, Julia 1.12.5, single core; treat them as ratios and re-run on the machine that matters to you.

The measurement method is part of the result. Naive loops disagree by nearly a factor of two, so the script fixes one method and states it: BenchmarkTools with the minimum sample; scalar draws accumulated with xor on the raw bits, because a floating-point + chain has four cycles of latency — longer than a draw from the fastest generators here, so it would measure the adder; nothing written to memory in the scalar loop, because storing every draw turns the benchmark into one of memory bandwidth (the same xoshiro reads 867 M/s with an accumulator and 481 M/s storing into an 8 MB vector); and `rand!` measured into a 4096-element vector, small enough to stay in cache.

Scalar draws: millions per second, and nanoseconds per draw. Every generator the package ships is listed, each scrambler variant separately. Run-to-run variation is a few percent, so read differences below that as noise.

Array fill with `rand!`, millions of elements per second (nanoseconds per element is 1000 over the figure):

What the tables say. Within each xoshiro family the ordering is the same and the spread is small: `+` is fastest, then `**`, then `++`, within about 15% of each other — the scrambler is a couple of instructions on top of a shared transition. Across families, state size costs: 128 and 256 bits are close, 512 bits is a third slower. The counter-based generators sit an order of magnitude below on `Float64`, which is what buying a keyed bijection per block costs, and the two 32-bit ciphers are fastest in `UInt32`, where a draw is one cipher word rather than two.

The two PCG variants land between the xoshiro families and the MRG generators, at roughly two thirds of Xoshiro256p: a 128-bit multiply per draw is more work than a handful of shifts and xors. PCG64DXSM is the faster of the two despite its extra output multiply, because its "cheap" 64-bit multiplier turns the 128x128 state multiplication into a 64x128 one. Both are the fastest generators in the package under `rand!`, ahead of

Generator	Float64	ns	UInt64	ns	UInt32	ns
MRG32k3a	236	4.25	47	21.06	95	10.48
MRG63k3a	196	5.10	93	10.76	199	5.04
Xoroshiro128p	697	1.43	941	1.06	1090	0.92
Xoroshiro128ss	655	1.53	876	1.14	848	1.18
Xoroshiro128pp	599	1.67	818	1.22	926	1.08
Xoshiro256p	788	1.27	1242	0.81	1221	0.82
Xoshiro256ss	715	1.40	1140	0.88	962	1.04
Xoshiro256pp	688	1.45	1032	0.97	1055	0.95
Xoshiro512p	570	1.75	798	1.25	742	1.35
Xoshiro512ss	523	1.91	742	1.35	614	1.63
Xoshiro512pp	532	1.88	725	1.38	665	1.50
PCG64	504	1.99	599	1.67	605	1.65
PCG64DXSM	561	1.78	743	1.35	745	1.34
Philox4x32-10	93	10.75	109	9.20	272	3.68
Philox4x64-10	203	4.94	262	3.82	259	3.86
Threefry4x32-20	84	11.86	92	10.89	186	5.39
Threefry4x64-20	147	6.80	153	6.52	157	6.35

Generator	Float64	UInt64	UInt32
MRG32k3a	163	47	94
MRG63k3a	184	93	185
Xoroshiro128p	546	581	581
Xoroshiro128ss	531	538	520
Xoroshiro128pp	507	500	511
Xoshiro256p	501	519	520
Xoshiro256ss	492	520	472
Xoshiro256pp	471	505	507
Xoshiro512p	380	391	426
Xoshiro512ss	375	393	390
Xoshiro512pp	377	389	408
PCG64	575	643	628
PCG64DXSM	632	744	735
Philox4x32-10	158	178	349
Philox4x64-10	359	339	257
Threefry4x32-20	100	106	206
Threefry4x64-20	209	181	156

every xoshiro: the whole state is one 128-bit word, so the bulk loop touches far less memory per element than a four- or eight-word state does.

The one outlier is MRG32k3a in UInt64, at 21.1 ns against 4.3 ns for its Float64: a 64-bit word costs four MRG steps, because the modulus is not a power of two and the word is assembled from 16-bit chunks. See the section on its integer outputs.

MRG63k3a is where that cost goes away, and the two rows read as one trade. Its Float64 is 20% slower (5.10 ns against 4.25) — the step is a 128-bit multiply where MRG32k3a's is a Float64 one — while its UInt64 is 2.0× faster (10.76 ns against 21.06) and its UInt32 2.1× faster (5.04 against 10.48), because a word is two 32-bit chunks rather than four 16-bit ones. Per random *bit* it is ahead everywhere: 0.081 ns against 0.133 for the double, where 63 bits come out of the same one step.

In bulk it is the recurrence-based generator that loses least: 184 against 196 million Float64/s is a drop of 6%, where MRG32k3a drops 31% and the xoshiro families about a third. Its UInt64 is flat at 93 either way. Why the drop is so much smaller here than for its sibling is not something these figures settle, so it is reported and not explained.

The counter-based generators are the ones that gain from filling an array: `rand!` produces whole blocks straight into it, so the counter, the block buffer and the index are touched once per block instead of once per draw. Philox4x64-10 goes from 203 to 359 million Float64 per second, a factor of 1.77. The recurrence-based generators have no such block to exploit and are *slower* in bulk than in the accumulator loop, by the cost of the stores.

Two combinations get no block path and fall back to the scalar loop: UInt32 from a 64-bit family, and any 64-bit draw taken while the generator sits mid-block because the caller has been mixing widths.

Every generator draws with zero allocations, in both paths, as do `short_jump!` and `long_jump!`.

Part VII

Validation

Chapter 7

Statistical Validation (TestU01)

`RandomDataStreams.jl` integrates seamlessly with [TestU01](#), the industry-standard C library for empirical statistical testing of RNGs, via the `RNGTest.jl` wrapper.

The validation is deliberately split in two. The **package test suite** runs `SmallCrush` only, over three suites, and answers the question "does this installation work" in about three minutes. **Crush and BigCrush run on demand** through `scripts/testu01/validate.jl`, because they take hours to days; see the last section.

7.1 In the test suite: `SmallCrush`

A subset of the TestU01 batteries, **SmallCrush**, is automatically executed on all supported generators (MRG32k3a, MRG63k3a, Xoshiro256+, PCG64, PCG64DXSM, Philox4x32-10, Philox4x64-10, Threefry4x64-20 and Threefry4x32-20) during the standard test suite. You can trigger this locally by running:

```
using Pkg
Pkg.test("RandomDataStreams")
```

7.2 Dependence between streams

A battery run on a single stream says nothing about whether the *streams* are independent of each other. Following L'Ecuyer et al. (2021, Sec. 6) — "it is also important to test the dependence between those streams [...] one can construct sequences that take a few values from each stream for a certain number of streams, in a round-robin fashion" — the test suite also runs `SmallCrush` on a sequence built by interleaving 64 streams, one value at a time, for each of the nine generators. For a counter-based generator this is what exercises the key schedule: a schedule handing out structurally related keys shows up here and not in a battery run on a single stream.

The heavier configurations recommended by the paper (up to 1024 streams and up to 8 values per stream) are in `scripts/testu01_bigcrush.jl`.

Platform limitation

The batteries do not run on **Apple Silicon**. `RNGTest` hands TestU01 its callback through `@cfunction($f, ...)`, which needs an executable trampoline for the closure, and macOS on aarch64 forbids memory that is both writable and executable — Julia raises `cfunction: closures are not supported on this platform`. This affects every generator and every battery, and is a property of the platform rather than of any package here.

The test suite probes the capability and skips the three batteries with a message when it is missing, so the rest of the suite still runs; the on-demand harness refuses with the same explanation. Everything else in the suite, including the reference vectors, runs everywhere.

7.3 The integer paths, at the bit level

The Crush batteries examine the 30 most significant bits of the $U(0, 1)$ outputs, so a battery run on the float says nothing about how a generator builds a machine integer. Where the integer *is* the native output — Philox 4x32, Threefry 4x32 — that is the same stream. Where it is a construction, it needs testing for itself, and `test/test_bits.jl` runs SmallCrush on the bit stream of: MRG32k3a in UInt32 and in UInt64 (16-bit chunks of a non-power-of-two modulus), MRG63k3a in the same two types (32-bit chunks of the same construction, with a modulus just under 2^{63}), the 64-bit counter-based families in UInt32 (the low half of a cipher word), PCG64 and PCG64DXSM in UInt32 (the low bits of a permuted output – DXSM ends in a multiplication, whose low bits depend on fewer input bits than its high ones), and Xoshiro256+ in UInt32 (the low bits of an additive scrambler).

This battery earns its place: a UInt64 construction for MRG32k3a that passed every $U(0, 1)$ battery failed here with six p-values at zero. Note also that passing does not clear the lowest bits of the + scramblers, whose linear weakness Blackman & Vigna document and SmallCrush does not resolve.

7.4 Running Crush and BigCrush

These are **not** part of `Pkg.test()` and must not become part of it. The package test suite answers "does this installation work", and SmallCrush at about a minute per battery is the most that belongs there. Crush takes hours and BigCrush the better part of a day per generator: that is a validation exercise, for a release or for a paper, not an installation check.

`scripts/testu01/validate.jl` runs any battery over any of the three suites, on demand:

```
# what would run, without running it
julia scripts/testu01/validate.jl --list --battery=bigcrush --suite=all

# one generator, one battery, one suite
julia scripts/testu01/validate.jl --battery=crush --generator=Xoshiro256p --suite=bits

# the full single-stream sweep
julia scripts/testu01/validate.jl --battery=bigcrush --suite=single
```

option	values	default
--battery	smallcrush, crush, bigcrush	smallcrush
--suite	single, interleaved, bits, all	single
--generator	a generator name, or all	all
--streams, --values	shape of the interleaved suite	64, 1
--out	directory for logs	testu01-results
--list	print the plan and exit	
--quiet	summary only, no per-test reports	

Each run writes TestU01's own report to its own log and appends a line to `summary.tsv` with the battery, suite, generator, wall time, Julia version and CPU. Per-test reports are on by default: they are the canonical artefact to archive, and they make a long run observable — the log fills as tests complete, so progress can be followed with `tail -f` and an interrupted run still leaves its completed tests behind. Run it under `nohup` or `tmux`:

```
nohup julia scripts/testu01/validate.jl --battery=bigcrush --suite=all > run.out 2>&1 &
tail -f testu01-results/bigcrush-*.log
```

7.5 Long campaigns: running for days without a session

validate.jl runs one battery in one process, serially. That is the wrong shape for a BigCrush sweep: the full matrix is seventeen generators times three suites, 51 runs, each taking the better part of a day, so a single process means weeks of sequential work in which one crash loses everything and a lost SSH session ends the campaign.

scripts/testu01/campaign.jl runs the same matrix as independent OS processes:

```
# what would run, and what is already done
julia scripts/testu01/campaign.jl --battery=bigcrush --status

# measure one job before committing to the sweep
julia scripts/testu01/campaign.jl --battery=bigcrush --calibrate

# the sweep, six jobs at a time
julia scripts/testu01/campaign.jl --battery=bigcrush --jobs=6
```

--battery is required: there is deliberately no default, so that a stray invocation cannot start a two-week sweep.

Resuming is the point. summary.tsv records one line per finished (battery, suite, generator). Re-running the same command skips those and does the rest, so an interruption costs only the jobs actually in flight — never the weeks already spent. Each job also writes its own summary.tsv in its own directory, which the driver reconciles, so even a driver killed mid-flight loses nothing that finished.

Stopping cleanly. touch <out>/STOP stops new jobs from starting and waits for the running ones, which is how to end a campaign without discarding a battery that is twenty hours in. Ctrl-C does the same.

Surviving logout and reboot. nohup survives a logout, tmux survives a logout and lets you reattach, and neither survives a reboot or restarts a driver that died. For a campaign measured in weeks, use the systemd *user* unit in scripts/testu01/randomdatastreams-campaign.service:

```
loginctl enable-linger $USER          # user services keep running after logout
cp scripts/testu01/randomdatastreams-campaign.service ~/.config/systemd/user/
$EDITOR ~/.config/systemd/user/randomdatastreams-campaign.service # set paths
systemctl --user daemon-reload
systemctl --user start randomdatastreams-campaign
journalctl --user -u randomdatastreams-campaign -f
```

The unit stops with SIGINT and a six-hour stop timeout, so systemctl --user stop lets the driver wait for its children rather than killing them, and Restart=on-failure brings back a driver that died — it resumes from summary.tsv like any other invocation.

When enable-linger is not yours to run

That first line is the one that needs an administrator. Without it, systemd kills user services at logout, which is exactly what a two-week sweep must survive. On a shared machine the answer is often "no", so scripts/run-campaign.sh can supervise the campaign with **tmux** instead:

```
./scripts/run-campaign.sh run --battery=crush --supervisor=tmux --watchdog
./scripts/run-campaign.sh status --battery=crush      # no flag needed afterwards
./scripts/run-campaign.sh stop  --battery=crush
```

--supervisor=auto, the default, uses systemd when linger is on or can be turned on, and falls back to tmux. The choice is recorded in the output directory, so status, stop and collect find the campaign without being told again.

What tmux does not give you is what --watchdog restores, through a *user* crontab entry that needs no privileges:

	systemd unit	tmux	tmux + --watchdog
survives logout	with linger	yes, unless KillUserProcesses=yes	same
survives a reboot	yes	no	yes (@reboot)
restarts a driver that died	Restart=on- failure	no	yes, within 15 min
attach and watch it work	journalctl -f	tmux attach	tmux attach

Two details the script handles, both of which are silent failures otherwise. KillUserProcesses=yes in logind.conf kills a detached tmux session at logout just as it kills an unlinger service, so tmux is *not* a workaround on such a machine — run-campaign.sh reads the setting and says so before you commit two weeks to it. And tmux keeps its socket under /run/user/\$UID, a directory that is removed when your last session ends; the script points TMUX_TMPDIR at a directory under \$HOME instead, so the session started from your login shell and the one the watchdog looks for are the same session.

The watchdog respects touch <out>/STOP: a campaign you stopped on purpose stays stopped. stop removes the crontab entry, and any crontab it edits is backed up to <out>/crontab.backup first — your other cron jobs are never touched.

A home directory that answers to a ticket

Where \$HOME is an NFS mount secured with Kerberos — sec=krb5 in the output of findmnt -T ~ -o OPTIONS — the files answer to a *ticket*, not to your uid. When the ticket lapses, every process you own loses the directory at the same instant: the driver in the middle of writing a result, and cron before it can so much as read the watchdog's copy of this script. Tickets are measured in hours and campaigns in days, so this is not a corner case. It is what happens by default to a sweep left to run.

Nothing in the campaign reports it, because nothing in the campaign can still write anywhere to report it. From outside, it looks like a watchdog that quietly stopped firing; the system's cron log tells the real story, one line per attempt:

```
CROND[91468]: (CRON) ERROR chdir failed (/u/you): Permission denied
```

prepare recognises the case. It refuses to begin without a valid ticket, and wraps the driver in krenew, which renews the ticket for as long as the campaign runs:

```
exec nice -n 10 ionice -c 3 /usr/bin/krenew -K 30 -- julia ... campaign.jl
```

Two limits are worth knowing before committing days to a sweep. First, **renewal has its own deadline**: a renewable ticket may be renewed only until the `renew` until that `klist -f` prints, typically some weeks after the `kinit` that created it, and a ticket that is not renewable at all cannot be rescued — `prepare` warns in both cases. A fresh login before a long campaign restarts that clock. Second, **the watchdog needs a ticket of its own**: the launcher's `krenew` runs only while the campaign does, and the watchdog exists for the times it does not. Keep one renewed independently of it:

```
krenew -K 30 -b          # a background daemon, renewing every 30 minutes
```

The alternative dissolves the question rather than answering it — put the checkout and the results on a filesystem that needs no ticket, such as local scratch space:

```
./scripts/run-campaign.sh prepare --battery=bigcrush --out=/var/tmp/$USER-bigcrush
```

Every result names the code that produced it. `summary.tsv` records the package version, the commit of HEAD and whether the working tree was clean, alongside the Julia version, the CPU and — for the interleaved suite — the number of streams and values per stream. Each battery log repeats it in its header. A campaign started from a dirty tree prints a warning before it begins, because a run measured in weeks cannot be repeated to find out afterwards which code produced it. The same line appears at the top of the throughput benchmark's report.

Do not benchmark and validate at the same time. The throughput figures in the implementation notes are minima over samples on an unloaded machine; a host running six BigCrush processes will produce numbers that measure the scheduler. Run `scripts/benchmarks/throughput.jl` first, pinned, then start the campaign. On a hybrid CPU (Intel 12th generation and later) pin the benchmark to a performance core with `taskset`; the batteries can use any core, since their results do not depend on how fast they were produced.

What still needs BigCrush

Two things the suite cannot settle at SmallCrush level:

- **The low bits of the + scramblers.** Blackman & Vigna document the lowest bits of `xoshiro256+` and `xoroshiro128+` as linearly weak. SmallCrush does not resolve it, so the `bits` suite passing at that level is not a clearance; the question needs Crush or BigCrush, and possibly does not show even there.
- **Dependence between streams at full strength.** The interleaved suite is the construction L'Ecuyer et al. (2021) call for, and the paper says batteries for counter-based generators still need development. A SmallCrush pass is a smoke test; the claim worth publishing is a BigCrush one.

Results belong in the JOSS submission, not in CI.

Part VIII

Generator Comparison

Chapter 8

Choosing a generator

RandomDataStreams.jl ships two kinds of generator. They differ in how a stream is defined, which is what makes them suited to different jobs — throughput is the smaller part of the choice.

Recurrence-based generators (MRG32k3a and MRG63k3a, the xoshiro/xoroshiro families) hold a state and a transition function. There is one orbit, and a stream is a *segment* of it: `next_stream!` jumps a fixed, huge distance along the orbit, and non-overlap is a statement about those distances.

Counter-based generators (Philox, Threefry) hold a counter and a key, and produce a block as a keyed bijection of the counter, $R = b_K(C)$. A stream is a *key*: different streams use different bijections, so their outputs cannot overlap by construction rather than by distance. Position within a stream is an index, not a state, so any draw can be recomputed from (key, substream, i) alone — see [Streams & Substreams](#).

	Recurrence-based	Counter-based
Stream =	a segment of one orbit	a distinct key
Non-overlap because	the jump distance exceeds any usable run	distinct keys are distinct bijections
Arbitrary jump	polynomial or matrix computation	one counter addition
Draw at index i	requires jumping there	computable directly
Speed	~5× faster per draw	block of N words amortises the bijection

8.1 Recurrence-based generators

(* means any of the +, **, ++ scramblers. PCG64DXSM has the same structure as PCG64, with a different output permutation and multiplier.)

8.2 Counter-based generators

Throughput figures come from the [Implementation Notes](#), which state the measurement method and the machine; read them as ratios.

8.3 When to choose MRG32k3a

- You need **provably non-overlapping streams with published, peer-reviewed parameters** (L'Ecuyer et al. 2002), e.g. to stay compatible with simulation libraries built on that model (SSJ, Simio, Arena-style stream ids...).

Property	MRG32k3a	MRG63k3a	PCG64	Xoroshiro128*	Xoshiro256*	Xoshiro512*
State (bits)	192 (6 × Int64)	384 (6 × Int64)	128	128	256	512
Period	$\approx 2^{191}$	$\approx 2^{377}$	2^{128}	$2^{128} - 1$	$2^{256} - 1$	$2^{512} - 1$
Native output	Float64 [0,1), ~32-bit resolution	Float64 (0,1), ~63-bit resolution	UInt64	UInt64	UInt64	UInt64
Float64 throughput	236 M/s	196 M/s	504 M/s (DXSM 561)	≈ 599 – 697 M/s	≈ 688 – 788 M/s	≈ 523 – 570 M/s
UInt64 throughput	47 M/s (4 steps)	93 M/s (2 steps)	599 M/s (DXSM 743)	≈ 818 – 941 M/s	≈ 1032 – 1242 M/s	≈ 725 – 798 M/s
Stream mechanism	matrix power $A^{2^{127}}$ on the seed	matrix power $A^{2^{250}}$ on the seed	closed-form LCG jump $(2^{32}(2^{64}+1)+1)$	long_jump! polynomial (2^{96})	long_jump! (2^{192})	long_jump! (2^{384})
Sub-stream mechanism	matrix power $A^{2^{76}}$	matrix power $A^{2^{150}}$	closed-form $(2^{64}+1)$	short_jump! (2^{64})	short_jump! (2^{128})	short_jump! (2^{256})
Backward jumps	yes (advance_state!) as forward	yes (advance_state!) as forward	yes, same cost as forward	yes (advance_state!) as forward	yes (advance_state!) as forward	yes (advance_state!) as forward
Arbitrary-jump cost	$O(e)$ matrix products	$O(e)$ matrix products	$O(\log n)$ multiplies	polynomials $O(\deg^2)$ GF(2) ops (~10–500 ms)	same	same
Equidistribution	well analysed theory	well analysed theory	none published	1-dim (**/++) / 0-dim (+)	3-dim (**/++) / 2-dim (+)	7-dim (**/++) / 6-dim (+)
Parallelism scale advised	large	very large	mild (2^{32} streams)	mild ($\approx 2^{32}$ streams)	very large	extreme

- Your application consumes **floats only**: MRG32k3a emits them natively. Its wide integers are assembled from 16-bit chunks and cost four MRG steps.
- You can afford $\sim 3\times$ slower generation than xoshiro.

8.4 When to choose MRG63k3a

Same author, same construction, same stream semantics, sized for 64-bit arithmetic (L'Ecuyer 1999, Table II, fourth entry):

- You want the **MRG stream model but your run consumes integers**. A UInt64 is two steps instead of four, and the 32-bit chunks it is built from depart from uniformity by 2^{-31} where MRG32k3a's 16-bit chunks depart by 2^{-16} .

Property	Philox4x32-10	Philox4x64-10	Threefry4x32-20	Threefry4x64-20
Block	4 × UInt32	4 × UInt64	4 × UInt32	4 × UInt64
Key (bits) → streams	64 → 2 ⁶⁴	128 → 2 ¹²⁸	128 → 2 ¹²⁸	256 → 2 ²⁵⁶
Counter (bits)	128	128	128	128
Substreams per stream	2 ⁶⁴	2 ⁶⁴	2 ⁶⁴	2 ⁶⁴
Words per substream	2 ⁶⁶ × 32 bits	2 ⁶⁶ × 64 bits	2 ⁶⁶ × 32 bits	2 ⁶⁶ × 64 bits
Arithmetic	wide multiply	wide multiply	add-rotate-xor only	add-rotate-xor only
Float64 throughput	93 M/s	210 M/s	84 M/s	147 M/s
Float64 via rand!	157 M/s	359 M/s	100 M/s	207 M/s
Jump to any position	O(1) counter arithmetic	O(1)	O(1)	O(1)
Output uniformity	exact (bijection)	exact	exact	exact
BigCrush	passes (Salmon et al. 2011)	passes	passes	passes

- You want the **longer period** — 2³⁷⁷ against 2¹⁹¹ — and the room it buys: 2¹²⁷ streams of 2¹⁰⁰ substreams of 2¹⁵⁰ numbers.
- You are **not** bound to the published MRG32k3a stream ids. MRG63k3a's jump distances are this package's own choice; nothing else implements them, so a run cannot be replayed stream-for-stream against another library.
- Prefer MRG32k3a if the run is float-heavy: on Float64 the 63-bit generator is about 17% slower (196 against 236 M/s), because the step costs a 128-bit multiply. It is ahead on every integer type — 93 against 47 M UInt64/s, 199 against 95 M UInt32/s, and 184 against 163 M Float64/s under rand! — and ahead per random *bit* everywhere.

8.5 When to choose a xoshiro/xoroshiro variant

- You want **maximum throughput** or raw 64-bit integers.
- Choose your state size by parallelism needs:
 - Xoroshiro128*: embedded/GPU or few workers only — its 2⁶⁴ substreams per 2³² streams are enough for mild parallelism. Note Xoroshiro128p has a mild Hamming-weight dependency after ~5 TB of output; prefer ss/pp.
 - Xoshiro256*: the sweet spot for general use (Julia itself uses xoshiro256++ as its default RNG). Use + if you generate *only* floating-point numbers from the high bits; otherwise prefer ++/**, whose outputs are fully equidistributed.
 - Xoshiro512*: essentially never needed for period reasons — any period ≥ 2²⁵⁶ is beyond every imaginable need — but handy when you want many more independent substreams than 2¹²⁸ without changing design.

8.6 When to choose PCG64

- You are **porting or checking a NumPy pipeline**. PCG64 is what `numpy.random.default_rng()` gives you, and the raw 64-bit outputs here match NumPy's exactly for the same state.

- You want the **fastest array fill** in the package: with a single 128-bit word of state, `rand!` reaches 632 M Float64/s for PCG64DXSM, ahead of every xoshiro. On scalar draws it is slower than xoshiro, at around 500 M/s.
- You need **cheap jumps to arbitrary positions**. The LCG advance is closed-form, so `advance_state!(rng, e, c)` costs $O(\log n)$ multiplications at any distance, forwards or backwards — against tens to hundreds of milliseconds for the equivalent xoshiro jump.
- Do **not** choose it for large-scale parallelism: with a 2^{128} period it offers 2^{32} streams, the same order as Xoroshiro128*, and there is no equidistribution theory for it. Note also that the stream distances here are odd rather than powers of two — for an LCG that is a correctness requirement, not a style choice; see the [Implementation Notes](#). For many streams, use MRG32k3a, a 256-bit xoshiro, or a counter-based generator.
- Note what this package does *not* do: PCG's own increment-based "streams". See the [FAQ](#).

8.7 When to choose Philox or Threefry

- You need to **address a draw without owning the generator that produced it**: a GPU kernel, a distributed worker, or a re-run that must reproduce replicate `i` of stream `s` without replaying anything. This is the property no recurrence-based generator has.
- You want a **very large number of streams** with no jump computation at all: `next_stream!` on a counter-based generator is an increment.
- You are matching another implementation of the same bijection — Random123, cuRAND, TensorFlow, Intel MKL all ship Philox, and the outputs here agree bit-for-bit with the Salmon et al. test vectors.
- Prefer **Philox4x64-10** as the default counter-based choice: it is the fastest of the four here and the widest-deployed. Prefer **Threefry** when the target hardware has no fast integer multiply, or when you want an add-rotate-xor construction for auditability; the same code reproduces bit-for-bit anywhere.
- Do not choose a counter-based generator for raw single-thread speed: on scalar Float64 draws they are 3–8× slower than xoshiro. Filling arrays with `rand!` recovers much of that (Philox4x64-10 gains a factor of 1.7) because a whole block lands in the array at once.

8.8 Scrambler choice within a xoshiro family

Scrambler	Output	Best for
+	sum of two state words	float-only workloads using the top bits; lowest 3 bits have low linear complexity
**	<code>rotl(s2·5, 7)·9</code>	all-purpose; maximal equidistribution (e.g. .NET/Lua default)
++	<code>rotl(s1+s4, k)+s1</code>	all-purpose; Rust's <code>SmallRng</code> , Julia's default

All three share the same transition and therefore the same jump constants: streams/substreams behave identically across scramblers of one family.

8.9 Related work in the Julia ecosystem

Most Julia random-number packages live under the [JuliaRandom](#) organisation, and they solve a different problem from this one. They are collections of *generators*; `RandomDataStreams.jl` is a *stream layer* that happens to need generators to apply itself to.

Package	What it provides	Relation to this package
Random-Numbers.jl	PCG, the xorshift family, MT19937 behind AbstractRNG	overlapping generators, no stream or substream operations; jump-ahead only for PCG (advance!)
Random123.jl	Philox, Threefry, ARS, AESNI	overlapping generators; counter positioning via <code>set_counter!</code> , no stream model
StableRNGs.jl	a Lehmer LCG with output stable across Julia versions	complementary; its "stable streams" mean version stability, not a partition of the period
RandomEx-tensions.jl	distribution objects and make specifications for <code>rand</code>	orthogonal: it extends what you draw, not where you draw it from
VSL.jl	bindings to Intel MKL's Vector Statistics Library	MKL offers <code>vslSkipAheadStream</code> and <code>vslLeapfrogStream</code> ; the bindings expose <code>vslNewStream</code> / <code>vslNewStreamEx</code> only
RNGTest.jl	the TestU01 interface	a dependency of ours — every battery in Validation runs through it

The practical consequence: if you need a fast generator, any of these will serve. If you need n streams that are provably disjoint, each rewindable to its own checkpoint, with the same four calls whatever the underlying family, that is what this package adds. Should you already depend on one of them, note that the generators here are validated against the same original references — the Salmon et al. test vectors for Philox and Threefry, the `xoshiro.di.unimi.it` C code for the xoshiro families, and NumPy for both PCG variants — so from an identical state they produce identical output. Seeding conventions differ between packages, however, so matching a run means setting the state, not reusing the seed.

8.10 Practical notes

- Seeding a recurrence-based generator: seed states must not be all-zero. For hash-like seeds, initialize with `splitmix64` outputs (see Vigna's recommendation). A counter-based generator has no forbidden key in this package — Philox and Threefry are safe for consecutive small keys — but a family whose bijection is weak for structured keys must supply a hashing `stream_key`; see [Implementation Notes](#).
- The package's jumps are **anchored at stream boundaries** ($Cg \leftarrow Bg \leftarrow Ig$), matching the L'Ecuyer stream model: `next_substream!` always lands at the same place regardless of how far you consumed in the current substream, which makes scenario replay reproducible. Counter-based generators obey the same contract, with the counter's high bits playing the role of `Bg`.
- Every generator here supports `advance_state!(rng, e, c)` with negative distances, so backward jumping is not a differentiator — only its cost is.
- Statistical testing of every generator is described in [Validation](#).

Part IX

FAQ

Chapter 9

FAQ

9.1 Why is an all-zero state forbidden?

For the xoshiro/xoroshiro families the all-zero state maps to itself: the generator would output only zeros forever. The two components of MRG32k3a and MRG63k3a have the same degeneracy (an all-zero component stays zero). Constructors, `srand!`, `set_state!` and `Random.seed!` all reject such a seed with an `ArgumentError`, and `checkseed` (`checkseed63` for MRG63k3a) is the predicate they use, so you can test a seed yourself before handing it over. An integer seed never produces one. PCG and the counter-based generators have no invalid seed at all.

9.2 Are MRG integer outputs uniform over their full width?

No, for neither member of the family, though the two are not equally far from it. MRG32k3a natively produces ~ 32 bits per draw ($z = p1 - p2$ in $[1, m1]$, $m1 = 2^{32} - 209$), and wider integers are assembled from 16-bit chunks of that value, whose departure from uniformity is of order 2^{-16} per chunk. MRG63k3a produces just under 63 bits ($m1 = 2^{63} - 6645$) and assembles 32-bit chunks, at 2^{-31} per chunk — the same construction, two orders of magnitude closer to uniform, and half as many steps per word.

Both are fine for indices, shuffling, flags and acceptance tests in simulations, and neither is exactly uniform over $2^{64}/2^{128}$. Use a xoshiro variant, PCG or a counter-based generator when you need full-width raw words.

9.3 MRG32k3a or MRG63k3a?

Same author, same construction, same stream semantics; the difference is the word size the arithmetic is built for. Take **MRG32k3a** when you need the published, widely implemented parameter set — matching SSJ, RngStreams or another simulation package draw for draw — or when your run consumes floats almost exclusively. Take **MRG63k3a** when the draws are integers (its `UInt64` costs two steps against four, and is far closer to uniform), or when you want the longer period (2^{377} against 2^{191}) and the wider stream spacing that comes with it. Note that the stream and substream distances of MRG63k3a are this package's own choice, since L'Ecuyer published jump matrices for MRG32k3a only.

9.4 Why doesn't `sample(v, k)` work with RandomDataStreams generators?

`sample` comes from StatsBase.jl, not from the Random standard library — even Julia's own RNGs do not provide it without that package. Everything in Random.jl works with RandomDataStreams generators.

9.5 Where did `srand`, `next_stream`, `short_jump`, `long_jump` go?

They mutated their argument despite not ending in `!`; they are deprecated in favour of:

Deprecated	Replacement
<code>srand(rng/gen, seed)</code>	<code>srand!(rng/gen, seed)</code>
<code>next_stream(gen)</code>	<code>next_stream!(gen)</code>
<code>short_jump(rng)</code>	<code>short_jump!(rng)</code>
<code>long_jump(rng)</code>	<code>long_jump!(rng)</code>

The old names still work but emit a deprecation warning.

9.6 What does an integer seed mean?

The same thing for every generator: it is expanded through `splitmix64` and folded into whatever that family accepts as a state or key, so `T(12345)`, `Gen(12345)` and `Random.seed!(T(), 12345)` all agree. A value in the family's own representation — its seed vector, or a `UInt128` for PCG — is taken as the state itself rather than hashed. See [Streams & Substreams](#) for the full table.

9.7 How do I run truly parallel simulations?

Take the streams first, then parallelise over them: `rngs = next_stream!(gen, nthreads())`. Streams share no state, so one per thread needs no synchronisation.

Never call `next_stream!` on a shared generator object from inside a parallel loop. The generator rewrites the seed of the next stream on every call, so concurrent calls hand out overlapping streams and nothing reports it. For separate processes the seeds have to travel instead; both patterns are in [Streams & Substreams](#).

9.8 Which generator should I pick?

See [Generator Comparison](#). Short answer: `Xoshiro256pp` for general use, `MRG32k3a` for L'Ecuyer-compatible stream semantics (`MRG63k3a` for the same model with 64-bit arithmetic and a longer period), `Philox4x64RNG` when a draw must be addressable by index rather than reached by replaying a state.

9.9 What does "counter-based" buy me?

A counter-based generator computes its output as a keyed bijection of a counter, so the draw at position `i` of substream `j` of stream `s` is a *function* of `(s, j, i)`. Nothing has to be replayed to reach it. That makes three things easy that are awkward otherwise: handing a stream identifier to a worker or a GPU kernel that has no generator object, jumping to an arbitrary position in constant time, and re-running one replicate of one stream without re-running anything else. The cost is speed: on scalar draws `Philox` and `Threefry` are several times slower than `xoshiro`, though filling arrays with `rand!` recovers much of the gap.

9.10 Philox or Threefry?

`Philox4x64RNG` is the faster of the two here and the one other libraries ship (`Random123`, `cuRAND`, `TensorFlow`, `Intel MKL`), so prefer it unless you have a reason not to. `Threefry4x64RNG` uses only additions, rotations and xors — no multiplication — which makes it a better fit for hardware without a fast wide multiply, and easier to audit.

9.11 Is it safe to use 1, 2, 3, ... as counter-based stream keys?

For Philox and Threefry, yes: ten (resp. twenty) rounds of encoding absorb the structure of consecutive small keys, as reported by Salmon et al. (2011) and discussed by L'Ecuyer et al. (2021, Sec. 3). This is *not* a property of counter-based generators in general — for the Squares generator, the key 0 gives an identically zero output — so a new family whose bijection is weak for structured keys must override `stream_key` with a schedule that hashes the seed. See [Implementation Notes](#).

9.12 Can I reproduce a NumPy pipeline?

Partly. PCG64 here produces exactly the same raw 64-bit outputs as `numpy.random.default_rng()`'s bit generator for the same 128-bit state, and PCG64DXSM matches NumPy's PCG64DXSM. What does *not* transfer is how NumPy gets to that state: `SeedSequence` turns a user seed into the state through its own hashing, and that specification is not implemented here. So you can reproduce a NumPy stream if you carry the state across, not if you carry only the seed.

The distributions differ too. `rand(rng)` and numpy's `random()` consume the same bits but not necessarily in the same way, and `randn` uses a different algorithm from NumPy's `ziggurat`. Only the raw generator output is guaranteed to agree.

9.13 Why can't I choose PCG's increment to get more streams?

Because it does not give independent streams. Every odd increment gives a full-period orbit, but for half of all pairs of odd increments the two orbits are the same state sequence shifted by a constant — an adversarial pair can be built whose outputs differ by an average of 2 bits out of 64. Most pairs are harmless, but no published criterion separates them, so the package fixes the increment and derives streams from jumps instead. The algebra is in the [Implementation Notes](#). NumPy takes the same practical position: its recommended way to parallelise is `SeedSequence.spawn()`, not the increment.

9.14 Does this package run on the GPU?

No. It assigns and navigates streams on the host. The counter-based bijections are pure functions of `(counter, key)`, so a device kernel can reproduce any draw the host assigned it from that pair alone; the generator objects themselves stay on the CPU. See [Streams & Substreams](#).